

ملخص شامل لـ

javascript

دليل تعليمي مختصر و منظم

Mohamed Amine

١١ نوفمبر ٢٠٢٥

جدول المحتويات

1. مدخل إلى جافاسكريبت — ما هي جافاسكريبت؟
2. مدخل إلى جافاسكريبت — كيف تعمل جافاسكريبت؟
3. مدخل إلى جافاسكريبت — بنية الجملة (Syntax)
4. مدخل إلى جافاسكريبت — أدوات المطور
5. مدخل إلى جافاسكريبت — التعليقات
6. أساسيات اللغة — المتغيرات (let, const)
7. أساسيات اللغة — أنواع البيانات
8. أساسيات اللغة — العمليات الحسابية
9. أساسيات اللغة — عمليات المقارنة والمنطق
10. أساسيات اللغة — التحويل بين الأنواع
11. التحكم في التدفق — الشروط (if, else if, else)
12. التحكم في التدفق — عبارة الاختيار (switch)
13. التحكم في التدفق — حلقات التكرار for
14. التحكم في التدفق — حلقات while و do...while
15. التحكم في التدفق — التحكم في الحلقات (break, continue)
16. الدوال والنطاق — تعريف الدوال واستدعاؤها
17. الدوال والنطاق — تمرير الوسائط وقيم الإرجاع
18. الدوال والنطاق — نطاق المتغيرات (Scope)
19. الدوال والنطاق — الرفع (Hoisting)

20. الدوال والنطاق — الدوال التعبيرية ودوال رد النداء
21. الكائنات الأساسية المضمنة — النصوص (Strings)
22. الكائنات الأساسية المضمنة — طرق النصوص (String Methods)
23. الكائنات الأساسية المضمنة — الأرقام والعمليات الرياضية (Numbers & Math)
24. الكائنات الأساسية المضمنة — التواريخ والأوقات (Dates)
25. الكائنات الأساسية المضمنة — تنسيق التواريخ
26. المصفوفات — تعريف المصفوفات
27. المصفوفات — الوصول للعناصر وتعديلها
28. المصفوفات — إضافة وحذف العناصر
29. المصفوفات — طرق المصفوفات (Array Methods)
30. المصفوفات — التكرار على المصفوفات (forEach, map, filter)
31. الكائنات وJSON — مقدمة إلى الكائنات
32. الكائنات وJSON — خصائص وطرق الكائنات
33. الكائنات وJSON — طرق الكائنات (Object Methods)
34. الكائنات وJSON — ترميز JSON
35. الكائنات وJSON — تحويل البيانات إلى JSON وبالعكس
36. ميزات ES6 الحديثة — تعريف المتغيرات (let, const)
37. ميزات ES6 الحديثة — دوال السهم (Arrow Functions)
38. ميزات ES6 الحديثة — القوالب النصية (Template Literals)
39. ميزات ES6 الحديثة — تفكيك المهيكلية (Destructuring)
40. ميزات ES6 الحديثة — الفئات (Classes)

41. ميزات ES6 الحديثة — المعاملات المنتشرة والمتبقية (Spread and Rest Operators)

42. التفاعل مع صفحة الويب (DOM) — مقدمة إلى DOM

43. التفاعل مع صفحة الويب (DOM) — الوصول للعناصر (Selecting Elements)

44. التفاعل مع صفحة الويب (DOM) — تعديل العناصر (Modifying Elements)

45. التفاعل مع صفحة الويب (DOM) — إضافة وحذف العناصر

46. التفاعل مع صفحة الويب (DOM) — الأحداث (Events)

47. التفاعل مع صفحة الويب (DOM) — مستمعو الأحداث (Event Listeners)

48. البرمجة غير المتزامنة — مقدمة إلى عدم التزامن

49. البرمجة غير المتزامنة — دوال التأخير الزمني (setTimeout, setInterval)

50. البرمجة غير المتزامنة — الوعود (Promises)

51. البرمجة غير المتزامنة — التعامل مع الوعود (then, catch, finally)

52. البرمجة غير المتزامنة — دوال async و await

53. التعامل مع الأخطاء والوحدات — التعامل مع الأخطاء (try, catch, finally)

54. التعامل مع الأخطاء والوحدات — رمي الأخطاء (throw)

55. التعامل مع الأخطاء والوحدات — الوحدات (Modules)

56. التعامل مع الأخطاء والوحدات — الاستيراد والتصدير (import/export)

مدخل إلى جافاسكريبت — ما هي جافاسكريبت؟

ما هي جافاسكريبت؟

مرحباً بك في عالم جافاسكريبت! في هذا الدرس الأول، سنتعرف على هذه اللغة البرمجية الأساسية التي أحدثت ثورة في الويب وجعلته حيويًا وتفاعليًا. ستكتشف كيف تضيف جافاسكريبت السلوك إلى صفحات الويب الثابتة.

المفهوم الأساسي: تعريف جافاسكريبت

جافاسكريبت (**JavaScript**) هي لغة برمجة خفيفة الوزن وتفسيرية (scripting language) تستخدم بشكل أساسي لجعل صفحات الويب تفاعلية. بفضلها، يمكن للمواقع أن تستجيب لإجراءات المستخدم، وتعرض محتوى ديناميكيًا، وتنفذ مهام معقدة مباشرة في متصفحك، مما يوفر تجربة مستخدم غنية وممتعة.

مثال عملي

```
console.log("أهلاً بك في عالم جافاسكريبت");
```

شرح الكود

يستخدم هذا الكود الأمر `console.log()` لطباعة رسالة نصية محددة بين علامتي التنصيص إلى وحدة التحكم (`console`) في المتصفح. إنها طريقة بسيطة للمبرمجين لرؤية مخرجات أكوادهم أو التحقق من قيم المتغيرات أثناء تطوير تطبيقات الويب، وهي خطوتك الأولى في التفاعل مع جافاسكريبت.

خلاصات الدرس

- جافاسكريبت هي لغة برمجة أساسية لجعل صفحات الويب تفاعلية وديناميكية.
- تعمل جافاسكريبت بشكل أساسي في متصفحات الويب (client-side) ويمكنها أيضاً العمل على الخوادم (server-side) باستخدام بيئات مثل Node.js.
- تضيف جافاسكريبت السلوك والتفاعل إلى الهيكل (HTML) والتصميم (CSS) للمواقع الإلكترونية.

مدخل إلى جافاسكريبت — كيف تعمل جافاسكريبت؟

كيف تعمل جافاسكريبت؟

توضح هذه النظرة السريعة كيف تعمل جافاسكريبت خلف الكواليس، بدءًا من كتابة الكود وحتى تنفيذه في بيئات مختلفة.

المحرك وبيئة التشغيل (Engine & Runtime)

جافاسكريبت هي لغة برمجة أحادية المسار (single-threaded)، مما يعني أنها تنفذ مهمة واحدة في كل مرة. يتم ذلك بواسطة **محرك جافاسكريبت** (مثل V8 في كروم وNode.js) الذي يقرأ الكود ويحوّله إلى تعليمات يفهمها الكمبيوتر. تحتاج جافاسكريبت أيضًا إلى **بيئة تشغيل** (مثل المتصفح أو Node.js) لتوفير واجهات برمجة التطبيقات (APIs) اللازمة للتفاعل مع العالم الخارجي مثل DOM في المتصفح أو نظام الملفات في Node.js.

مثال على الكود

```
let greeting = "أهلاً بك في عالم جافاسكريبت";
console.log(greeting);

function sum(a, b) {
  return a + b;
}

let result = sum(5, 3);
console.log("المجموع هو " + result);
```

شرح الكود

في هذا المثال، يعرف الكود متغيرًا نصيًا (greeting) ثم يطبعه باستخدام console.log. بعد ذلك، يتم تعريف دالة بسيطة تسمى sum تقوم بجمع عددين. أخيرًا، يتم استدعاء الدالة sum مع قيمتين، وتُخزّن النتيجة في متغير result، ثم يتم طباعة هذا المتغير. كل سطر من هذا الكود يتم قراءته وتفسيره وتنفيذه بالتسلسل بواسطة محرك جافاسكريبت في بيئة التشغيل.

- جافاسكريبت أحادية المسار، مما يعني أنها تنفذ عملية واحدة في كل مرة.
- **محرك جافاسكريبت** (مثل V8) يقرأ ويفسر الكود الخاص بك.
- تحتاج جافاسكريبت إلى **بيئة تشغيل** (مثل المتصفح أو Node.js) لتعمل وتتفاعل مع العالم الخارجي.

مدخل إلى جافاسكريبت — بنية الجملة (Syntax)

بنية الجملة (Syntax) في جافاسكريبت

بنية الجملة هي مجموعة القواعد الأساسية التي يجب اتباعها عند كتابة الشيفرة البرمجية بلغة جافاسكريبت. فهم هذه القواعد أمر حيوي لتمكين متصفح الويب أو بيئة التشغيل من فهم وتنفيذ الأوامر التي تكتبها بشكل صحيح. أي خطأ في البنية سيؤدي إلى عدم عمل البرنامج.

القواعد الأساسية لبنية جافاسكريبت

تتبع جافاسكريبت قواعد بسيطة لتنظيم الشيفرة وجعلها قابلة للقراءة والتنفيذ: تنتهي معظم الجمل البرمجية (statements) بفاصلة منقوطة (;). تُستخدم الأقواس المعقوفة ({}) لتجميع الأوامر في كتل برمجية، مثل وظائف أو حلقات. جافاسكريبت حساسة لحالة الأحرف (case-sensitive)، مما يعني أن myVar تختلف عن myvar.

مثال على الكود

```
let userName = "أحمد";
const age = 30;
let message = "مرحباً يا " + userName + "!";
console.log(message);
console.log("سنة " + age + ":عمرك هو");
```

شرح الكود

في هذا المثال، قمنا بتعريف متغيرين: userName باستخدام let لتخزين قيمة نصية، وage باستخدام const لتخزين قيمة رقمية ثابتة. لاحظ أن كل جملة تنتهي بفاصلة منقوطة. ثم قمنا بإنشاء متغير message لدمج النصوص، واستخدمنا الدالة console.log لعرض الرسائل وقيمة العمر في وحدة التحكم (console)، مما يوضح كيفية دمج المتغيرات مع النصوص.

- **إنهاء الجمل (;):** معظم الأوامر في جافاسكريبت تنتهي بفاصلة منقوطة، وهذا يحدد نهاية الجملة.
- **الكتل البرمجية ({}):** تُستخدم لتجميع مجموعة من الأوامر معًا، مثل داخل وظيفة أو شرط.
- **حساسية حالة الأحرف:** يجب الانتباه إلى الأحرف الكبيرة والصغيرة؛ ف MyVar ليست هي نفسها myVar.

مدخل إلى جافاسكريبت — أدوات المطور

أدوات المطور

تعد أدوات المطور (Developer Tools) مجموعة أساسية من الأدوات المضمنة في متصفحات الويب. تساعد هذه الأدوات المبرمجين على فحص عناصر الصفحة، تصحيح الأخطاء البرمجية (Debugging)، تحليل أداء تطبيقات الويب، وفهم كيفية تفاعل جافاسكريبت مع الصفحة.

وحدة التحكم (Console)

وحدة التحكم هي نافذة ضمن أدوات المطور تسمح لك برؤية رسائل من جافاسكريبت، تنفيذ أوامر برمجية مباشرة، وعرض الأخطاء والتحذيرات. إنها الأداة الأولى التي يستخدمها المطورون الجدد لفهم وتصحيح أكوادهم.

مثال عملي

```
let userName = "أحمد";  
console.log("إني عالم جافاسكريبت" + userName + " مرحباً بك يا " + userName);
```

شرح الكود

في هذا المثال، قمنا أولاً بتعريف متغير اسمه `userName` وإسناد القيمة "أحمد" إليه. ثم استخدمنا الدالة `console.log()` لعرض رسالة ترحيب مخصصة في وحدة التحكم. هذه الدالة مفيدة جداً لطباعة القيم المختلفة للمتغيرات ومتابعة تدفق تنفيذ الكود.

- أدوات المطور هي جزء لا يتجزأ من متصفح الويب وتساعد في عملية تطوير الويب.
- تعتبر وحدة التحكم (Console) الأداة الأكثر استخداماً لعرض المخرجات وتصحيح الأخطاء في جافاسكريبت.
- يمكنك استخدام `console.log()` لطباعة الرسائل والمتغيرات إلى وحدة التحكم لتتبع ما يفعله الكود.

مدخل إلى جافاسكريبت — التعليقات

التعليقات في جافاسكريبت

في البرمجة، الكود الجيد ليس فقط ما يعمل، بل ما يمكن فهمه وصيانته. التعليقات هي ملاحظات نصية يمكنك إضافتها إلى كود جافاسكريبت الخاص بك لتوضيح أجزاء منه. يتجاهلها المتصفح تمامًا عند تنفيذ البرنامج، مما يعني أنها لا تؤثر على طريقة عمل الكود.

أنواع التعليقات

التعليقات أداة أساسية لتوثيق الكود. هناك نوعان رئيسيان في جافاسكريبت:

- **تعليق السطر الواحد:** يبدأ ب `//` ويستمر حتى نهاية السطر.
- **تعليق الأسطر المتعددة:** يبدأ ب `/*` وينتهي ب `*/`، ويمكن استخدامه لتعليق كتلة من الكود أو نص طويل.

مثال عملي

```
let studentName = "سارة";
let score = 95;
let message = "النتيجة: " + studentName + " : اسم الطالبة ";
console.log(message);

let itemPrice = 50;
let discount = 0.10; // 10% discount in decimal
let finalPrice = itemPrice - (itemPrice * discount);
console.log("السعر النهائي: " + finalPrice);
```

شرح الكود

يعرض هذا الكود تعريف المتغيرات، دمج النصوص، وإجراء عملية حسابية بسيطة لحساب سعر نهائي بعد خصم. في سياق هذا الدرس، يمكننا تخيل كيف ستجعل التعليقات فهم هذه الأسطر أسهل، كأن نضيف ملاحظة تشرح الغرض من متغير `score` أو العملية الحسابية لـ `finalPrice`.

نقاط رئيسية

- **للتوثيق والفهم:** التعليقات هي ملاحظات للمبرمجين ولا يقوم محرك جافاسكريبت بتنفيذها.

- **تحسين قابلية القراءة:** تجعل الكود أسهل في القراءة، الفهم، والصيانة للمطورين الآخرين أو لنفسك في المستقبل.
- **تحديد الأنواع:** استخدم // لسطر واحد، و /* ... */ لعدة أسطر من التعليقات.

أساسيات اللغة — المتغيرات (let, const)

المتغيرات (let, const) في JavaScript

في JavaScript، المتغيرات هي أماكن لتخزين البيانات. أصبحت `let` و `const` هي الطرق الحديثة والمفضلة لتعريف المتغيرات، حيث توفران تحكمًا أفضل في نطاق المتغيرات وقابليتها للتعديل مقارنة بـ `var` القديمة.

مفهوم المتغيرات (let, const)

تُستخدم **let** لتعريف متغير يمكن إعادة تعيين قيمته لاحقًا. أما **const** فتُستخدم لتعريف ثابت (constant)، أي متغير لا يمكن تغيير قيمته بعد التعيين الأولي له.

مثال عملي

```
let userName = "أحمد";
console.log(userName); // Output: أحمد

userName = "سارة"; // يمكن تغيير قيمة
console.log(userName); // Output: سارة

const PI = 3.14;
console.log(PI); // Output: 3.14

// PI = 3.14159; // محاولة تغيير قيمة
// ستؤدي إلى خطأ
```

شرح الكود

في هذا المثال، قمنا أولاً بتعريف متغير `userName` باستخدام `let` وأسندنا له قيمة "أحمد"، ثم غيرنا قيمته إلى "سارة" لاحقًا بنجاح. أما الثابت `PI` فقد تم تعريفه بـ `const` بقيمة 3.14، ومحاولة إعادة تعيين قيمة جديدة له ستفشل وتنتج خطأ لأن `const` يمنع تغيير القيمة بعد التعيين الأولي.

- **let** تُستخدم للمتغيرات التي قد تتغير قيمتها لاحقًا.
- **const** تُستخدم للقيم الثابتة التي لا يجب أن تتغير أبدًا بعد تعريفها.
- اختيار `let` أو `const` يعتمد على ما إذا كنت تحتاج إلى تغيير قيمة المتغير أم لا.

أساسيات اللغة — أنواع البيانات

أنواع البيانات في JavaScript

في لغة البرمجة JavaScript، تعد البيانات اللبنة الأساسية لكل تطبيق. لفهم كيفية معالجة هذه البيانات وتخزينها بكفاءة، من الضروري معرفة أنواعها المختلفة. هذا الدرس يقدم لك مقدمة سريعة لأنواع البيانات الأساسية.

ما هي أنواع البيانات؟

أنواع البيانات (Data Types) هي تصنيفات للقيم التي يمكن للغة JavaScript التعامل معها. تحدد هذه الأنواع طبيعة القيمة ونوع العمليات التي يمكن إجراؤها عليها، مما يضمن تنظيمًا وفعالية أكبر في الكود.

مثال عملي

```
let userName = "سارة"; // String (نص)
let userAge = 25;      // Number (رقم)
let isLoggedIn = true; // Boolean (قيمة منطقية)
let carBrand = null;   // null (لا قيمة)
let favoriteColor;     // undefined (غير معرف)
let userHobbies = ["الرياضة", "الرسم"]; // Object (مصفوفة)

console.log(typeof userName);
console.log(typeof userAge);
console.log(typeof isLoggedIn);
console.log(typeof carBrand);
console.log(typeof favoriteColor);
console.log(typeof userHobbies);
```

شرح الكود

يوضح هذا المثال كيفية تعريف متغيرات بأنواع بيانات مختلفة. المتغير `userName` يخزن نصًا، و `userAge` يخزن رقمًا، و `isLoggedIn` يخزن قيمة منطقية (صحيح/خطأ). المتغير `carBrand` لا يحمل أي قيمة (`null`)، و `favoriteColor` لم يُسند له قيمة بعد (`undefined`). أخيرًا، `userHobbies` هو مصفوفة تحتوي على عدة قيم، وتعتبر من نوع `object`. تُظهر أوامر `console.log(typeof ...)` نوع كل متغير.

- **أهمية التحديد:** تساعد أنواع البيانات JavaScript على فهم كيفية تخزين ومعالجة القيم.

- **الأنواع الأساسية:** تشمل النص (String)، الرقم (Number)، والقيم المنطقية (Boolean).
- **معرفة النوع:** يمكن استخدام العامل typeof لمعرفة نوع بيانات أي متغير أو قيمة.

أساسيات اللغة — العمليات الحسابية

العمليات الحسابية في JavaScript

تُعد العمليات الحسابية جزءاً أساسياً في أي لغة برمجة، وتُمكنك JavaScript من إجراء جميع العمليات الرياضية الأساسية على الأرقام. فهم هذه العمليات ضروري لبناء أي منطق يعتمد على الحسابات.

أنواع العمليات الحسابية الأساسية

تدعم JavaScript العمليات الحسابية الشائعة مثل الجمع والطرح والضرب والقسمة، بالإضافة إلى عملية باقي القسمة، التي تُعد مهمة في العديد من السيناريوهات البرمجية.

مثال توضيحي

```
let num1 = 20;
let num2 = 6;

let sum = num1 + num2; // الجمع
let difference = num1 - num2; // الطرح
let product = num1 * num2; // الضرب
let quotient = num1 / num2; // القسمة
let remainder = num1 % num2; // باقي القسمة

console.log("الجمع: " + sum);
console.log("الطرح: " + difference);
console.log("الضرب: " + product);
console.log("القسمة: " + quotient);
console.log("باقي القسمة: " + remainder);
```

شرح الكود

في هذا المثال، قمنا بتعريف متغيرين رقميين هما `num1` و `num2`. ثم أجرينا عليها عمليات الجمع (+)، الطرح (-)، الضرب (*)، والقسمة (/). وأخيراً، استخدمنا عامل باقي القسمة (%) الذي يعطينا الباقي بعد إجراء عملية القسمة الصحيحة. يتم عرض نتائج كل عملية باستخدام `console.log`.

- تستخدم JavaScript رموزاً قياسية للعمليات الحسابية مثل +، -، *، /.

- يُستخدم عامل % (باقي القسمة) للحصول على الباقي بعد قسمة عدد على آخر.
- العمليات الحسابية ضرورية لمعالجة البيانات وبناء المنطق الرياضي في تطبيقاتك.

أساسيات اللغة — عمليات المقارنة والمنطق

عمليات المقارنة والمنطق في JavaScript

تُعد عمليات المقارنة والمنطق أدوات أساسية في JavaScript لتمكين برامجنا من اتخاذ القرارات الذكية. فباستخدامها، يمكننا فحص الشروط المختلفة وتحديد مسار سير البرنامج بناءً على ما إذا كانت هذه الشروط صحيحة أم خاطئة.

مقارنة القيم واتخاذ القرارات

تُستخدم **عمليات المقارنة** لفحص العلاقة بين قيمتين، مثل المساواة (`==`, `===`)، عدم المساواة (`!=`, `!==`)، أكبر من (`>`)، أصغر من (`<`)، وهكذا. أما **العمليات المنطقية** (مثل `&&` "و"، `||` "أو"، `!` "ليس") فتسمح لنا بدمج عدة شروط معًا أو عكسها لإنشاء شروط أكثر تعقيدًا.

مثال عملي

```
let temperature = 25;
let isSunny = true;
let isDaytime = true;

// عملية مقارنة ومنطق
let goToPark = (temperature > 20 && isSunny) || (isDaytime && temperature < 30);

console.log("هل يجب أن نذهب إلى المتنزه؟", goToPark);

let isRaining = false;
let stayHome = !isSunny || isRaining;

console.log("هل يجب أن نبقى في المنزل؟", stayHome);
```

شرح الكود

في هذا المثال، نقوم بتعريف ثلاث متغيرات: درجة الحرارة، وما إذا كان الطقس مشمسًا، وما إذا كان الوقت نهارًا. المتغير `goToPark` يستخدم عملية منطقية معقدة: يذهب إلى المتنزه إذا كانت درجة الحرارة أكبر من 20 و الطقس مشمس، أو إذا كان الوقت نهارًا و درجة الحرارة أقل من 30. أما المتغير `stayHome` فيستخدم عملية "ليس" (`!`) لعكس قيمة `isSunny` ويجمعها مع `isRaining` باستخدام "أو" (`||`).

- **المقارنة:** تفحص العلاقة بين قيمتين (مثلاً < للتحقق مما إذا كانت قيمة أكبر من أخرى).
- **المنطق:** يربط أو يعكس الشروط لتكوين تعابير أكثر تعقيدًا (مثل && تعني "و"، || تعني "أو"، ! تعني "ليس").
- **النتيجة:** دائمًا ما تكون ناتج هذه العمليات إما **true** (صحيح) أو **false** (خاطئ).

أساسيات اللغة — التحويل بين الأنواع

التحويل بين الأنواع في JavaScript

في JavaScript، قد تحتاج أحيانًا لتغيير نوع البيانات لقيمة ما، مثل تحويل نص إلى رقم أو العكس. هذه العملية تسمى **التحويل بين الأنواع** (Type Conversion)، وهي أساسية لفهم كيفية تفاعل البيانات المختلفة في برامجك وكيفية التعامل معها بشكل صحيح.

مفهوم التحويل بين الأنواع

هو عملية تغيير نوع بيانات قيمة معينة (مثلًا من نص **string** إلى رقم **number**). يمكن أن يتم هذا التحويل بشكل **صریح** (عندما تطلب أنت من JavaScript تغيير النوع باستخدام دالة معينة) أو **ضمني** (عندما تقوم JavaScript بالتغيير تلقائيًا في سياقات معينة مثل العمليات الحسابية).

مثال عملي

```
let priceText = "150.75";
let quantity = 5;

// تحويل نص إلى رقم
let price = Number(priceText); // price will be 150.75 (number)

// إجراء عملية حسابية
let total = price * quantity; // total will be 753.75

// تحويل رقم إلى نص
let totalMessage = "الإجمالي: " + String(total); // totalMessage will be "753.75: الإجمالي"

let invalidText = "hello";
let invalidNum = Number(invalidText); // invalidNum will be NaN (Not a Number)
```

شرح الكود

في هذا المثال، نستخدم الدالة `Number()` لتحويل النص "150.75" إلى قيمة رقمية يمكن استخدامها في العمليات الحسابية. بعد حساب الإجمالي، قمنا بتحويل الرقم `total` إلى نص باستخدام الدالة `String()` لدمجه مع

نص آخر وعرضه. لاحظ أن محاولة تحويل نص غير رقمي مثل "hello" باستخدام Number() ينتج عنها NaN (اختصار لـ "ليس رقمًا")، مما يشير إلى فشل التحويل.

- يمكنك إجراء التحويل **الصريح** بين الأنواع باستخدام دوال مثل String()، Number()، و Boolean().
- فهم كيفية عمل التحويل الضمني (Coercion) سيساعدك على تجنب الأخطاء غير المتوقعة في برامجك.
- انتبه لنتائج التحويل غير الصالحة، فمحاولة تحويل نص لا يمثل رقمًا إلى رقم ستعطي القيمة NaN.

التحكم في التدفق — الشروط (if, else if, else)

الشروط (if, else if, else)

تخيل أن برنامجك يحتاج لاتخاذ قرارات بناءً على مواقف مختلفة. هنا يأتي دور **الشروط**! تسمح لك الشروط بالتحكم في تدفق الكود، حيث يتم تنفيذ أجزاء معينة من الكود فقط إذا كانت شروط محددة صحيحة. هذا يمنح برنامجك القدرة على أن تكون أكثر ذكاءً وتفاعلية.

مفهوم الشروط

تُعد عبارات `if`، `else if`، و `else` أدوات أساسية لتوجيه برنامج JavaScript. تسمح لك هذه الأدوات بتحديد مسار مختلف لتنفيذ الكود بناءً على ما إذا كانت الشروط التي تحددها صحيحة (`true`) أم خاطئة (`false`).

مثال عملي

```
let score = 75;

if (score >= 90) {
  console.log("ممتاز! حصلت على تقدير");
} else if (score >= 70) {
  console.log("B. جيد جداً! حصلت على تقدير");
} else {
  console.log("C. أو أقل تحتاج إلى تحسين. تقدير");
}
```

شرح الكود

في هذا المثال، نحدد متغير `score` بقيمة 75. يبدأ البرنامج بفحص الشرط الأول في `if`: هل `score` أكبر من أو يساوي 90؟ بما أن الإجابة **لا** (75 ليست أكبر من 90)، ينتقل إلى فحص الشرط في `else if`: هل `score` أكبر من أو يساوي 70؟ بما أن الإجابة **نعم** (75 أكبر من 70)، سيتم تنفيذ الكود داخل `else if` وطباعة "جيد جداً! حصلت على تقدير B.". ويتجاهل باقي الشروط.

- تُستخدم `if` لتنفيذ كود معين إذا كان الشرط صحيحاً.
- تُستخدم `else if` لفحص شروط إضافية إذا كان الشرط السابق خاطئاً.
- تُستخدم `else` لتنفيذ كود معين إذا كانت جميع الشروط السابقة (`if` و `else if`) خاطئة.

التحكم في التدفق — عبارة الاختيار (switch)

عبارة الاختيار (switch) في JavaScript

تُعد **عبارة الاختيار (switch)** أداة قوية في JavaScript للتحكم في تدفق البرنامج. تسمح لك بتنفيذ كتل مختلفة من التعليمات البرمجية بناءً على قيمة متغيرة واحدة، مما يجعل الكود أكثر تنظيماً ووضوحاً من استخدام العديد من عبارات if...else if المتسلسلة.

المفهوم الأساسي

تقوم عبارة switch بتقييم تعبير مرة واحدة ثم تقارن نتيجة هذا التعبير بقيم محددة (case). عند العثور على تطابق، يتم تنفيذ الكود المرتبط بتلك الحالة. وإذا لم يتم العثور على تطابق، يمكن تنفيذ كود افتراضي (default).

مثال الكود

```
let dayNumber = 3;
let dayName;

switch (dayNumber) {
  case 1:
    dayName = "الأحد";
    break;
  case 2:
    dayName = "الاثنين";
    break;
  case 3:
    dayName = "الثلاثاء";
    break;
  case 4:
    dayName = "الأربعاء";
    break;
  default:
    dayName = "يوم غير معروف";
}

console.log("اليوم هو:" + dayName);
```

شرح الكود

في هذا المثال، نقوم بتحديد قيمة للمتغير `dayNumber`. نستخدم عبارة `switch` هذه القيمة للمقارنة مع الحالات المختلفة (`case`). عندما تتطابق القيمة (في هذه الحالة 3 مع `case 3`)، يتم تنفيذ الكود الخاص بتلك الحالة وتعيين "الثلاثاء" للمتغير `dayName`. تُنهي كلمة `break` تنفيذ عبارة `switch` لمنع "السقوط" إلى الحالات التالية، ثم يتم طباعة اسم اليوم.

- تستخدم **عبارة switch** لاختيار مسار تنفيذ واحد من عدة مسارات محتملة بناءً على قيمة تعبير واحد.
- كلمة **break** ضرورية داخل كل `case` لمنع تنفيذ الكود الخاص بالحالات التالية (`fall-through`).
- يمكنك استخدام كلمة **default** لتعريف كتلة كود يتم تنفيذها إذا لم تتطابق أي من حالات `case` مع قيمة التعبير.

التحكم في التدفق — حلقات التكرار for

حلقات التكرار for في JavaScript

تُعد حلقات التكرار (Loops) أدوات قوية في البرمجة لتنفيذ نفس الكتلة من التعليمات عدة مرات. حلقة **for** هي واحدة من أكثر الحلقات شيوعًا واستخدامًا للتحكم في تدفق البرنامج عندما نعرف عدد مرات التكرار مسبقًا.

مفهوم حلقة التكرار for

تسمح لك حلقة **for** بتكرار كتلة من التعليمات لعدد محدد من المرات. تتكون من ثلاثة أجزاء رئيسية: تهيئة متغير العداد، شرط الاستمرار في التكرار، وتحديث العداد بعد كل دورة.

مثال على الكود

```
for (let i = 0; i < 3; i++) {  
  console.log("العد الحالي هو " + i);  
}
```

شرح الكود

يبدأ هذا الكود بتهيئة العداد **i** بالقيمة صفر (`let i = 0`). يستمر في التكرار طالما أن **i** أصغر من 3 (`i < 3`). في كل تكرار، يقوم بطباعة قيمة **i** الحالية ثم يزيد **i** بمقدار واحد (`++i`). لذلك، ستتم طباعة "العد الحالي هو: 0" ثم "العد الحالي هو: 1" وأخيرًا "العد الحالي هو: 2" قبل أن تتوقف الحلقة.

نقاط رئيسية

- حلقة **for** مثالية لتكرار التعليمات لعدد معروف مسبقًا من المرات.
- تتكون من تهيئة، شرط، وتحديث، وهي أساسية للتحكم الفعال في تدفق البرنامج.
- تساعد في كتابة كود أكثر كفاءة وتجنب تكرار نفس الأسطر يدويًا.

التحكم في التدفق — حلقات while و do...while

التحكم في التدفق: حلقات while و do...while في JavaScript

تُعد حلقات التكرار أدوات أساسية في البرمجة لتنفيذ كتلة من التعليمات البرمجية عدة مرات. في جافاسكريبت، تُستخدم حلقتا **while** و **do...while** لتكرار الكود بناءً على شرط معين، مما يساعد في أتمتة المهام المتكررة.

حلقة while

تُنفذ حلقة **while** مجموعة من التعليمات البرمجية طالما أن الشرط المحدد صحيح. تقوم هذه الحلقة بفحص الشرط **قبل** كل عملية تكرار. إذا كان الشرط خاطئًا من البداية، فلن يتم تنفيذ الكود داخل الحلقة مطلقًا.

مثال الكود (while)

```
let counter = 0;
while (counter < 3) {
  console.log("العداد هو " + counter);
  counter++;
}
```

شرح الكود

في هذا المثال، تبدأ قيمة المتغير **counter** من الصفر. تستمر الحلقة في طباعة قيمة **counter** وزيادتها بواحد طالما أن **counter** أقل من 3. ستطبع الأعداد 0 و 1 و 2، ثم تتوقف.

حلقة do...while

تشبه حلقة **do...while** حلقة **while**، ولكنها تضمن تنفيذ كتلة التعليمات البرمجية **مرة واحدة على الأقل**. يتم فحص الشرط **بعد** تنفيذ الكود لأول مرة، وهذا هو الفارق الرئيسي.

مثال الكود (do...while)

```
let i = 0;
do {
  console.log("القيمة هي " + i);
}
```



```
i++;  
} while (i < 3);
```

شرح الكود

هنا، يتم تنفيذ الكود داخل do أولاً (حتى لو كان الشرط خاطئاً، سيتم تنفيذه مرة واحدة)، ثم يتم فحص الشرط $i < 3$. سيتم طباعة الأعداد 0 و 1 و 2، حيث تكرر الحلقة طالما الشرط صحيح.

- **حلقة while:** تفحص الشرط قبل التنفيذ؛ قد لا تعمل أبداً إذا كان الشرط خاطئاً من البداية.
- **حلقة do...while:** تنفذ الكود مرة واحدة على الأقل، ثم تفحص الشرط.
- كلتا الحلقتين تُستخدمان لتكرار الكود بناءً على شرط، وهما مفيدتان عندما لا تعرف عدد التكرارات مسبقاً.

التحكم في التدفق — التحكم في الحلقات (break, continue)

التحكم في الحلقات (break, continue)

في لغة JavaScript، تسمح لنا الحلقات بتكرار تنفيذ مجموعة من الأوامر. لكن في بعض الأحيان، نحتاج إلى تغيير هذا التدفق الطبيعي للحلقة، سواء بإيقافها تمامًا أو بتخطي دورة معينة. هنا يأتي دور الكلمتين المفتاحيتين **break** و **continue**.

break

تُستخدم الكلمة **break** للخروج من الحلقة فورًا. بمجرد أن يتم تنفيذها داخل حلقة، تتوقف الحلقة تمامًا ويستمر تنفيذ الكود الذي يلي الحلقة مباشرة.

مثال على break

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    break;  
  }  
  console.log("الرقم:", i);  
}  
  
// الناتج:  
// الرقم: 1  
// الرقم: 2
```

شرح الكود

في هذا المثال، تستمر الحلقة في الطباعة طالما أن *i* ليس 3. عندما يصبح *i* مساويًا لـ 3، يتم تنفيذ **break** وتتوقف الحلقة بالكامل، فلا تتم طباعة 3 أو 4 أو 5.

continue

تُستخدم الكلمة **continue** لتخطي الدورة الحالية للحلقة والانتقال مباشرة إلى الدورة التالية. أي أنها تتجاهل بقية التعليمات في الكتلة الحالية وتنتقل إلى تحديث شروط الحلقة.

مثال على continue

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  console.log("الرقم:", i);  
}  
  
// الناتج:  
// الرقم: 1  
// الرقم: 2  
// الرقم: 4  
// الرقم: 5
```

شرح الكود

هنا، عندما يصبح i مساويًا لـ 3، يتم تنفيذ `continue`. هذا يعني أن أمر الطباعة لـ $i = 3$ يتم تخطيه، وتنتقل الحلقة مباشرة إلى الدورة التالية حيث يصبح $i = 4$ ، وهكذا.

ملخص سريع

- **break**: ينهي الحلقة بالكامل فورًا عند تنفيذه.
- **continue**: يتخطى الدورة الحالية للحلقة وينتقل إلى الدورة التالية.
- تمنحك هذه الكلمات تحكمًا دقيقًا ومرنًا في تدفق تنفيذ حلقاتك البرمجية.

الدوال والنطاق — تعريف الدوال واستدعاؤها

تعريف الدوال واستدعاؤها

الدوال (Functions) هي كتل برمجية تؤدي مهمة محددة. تسمح لك بتنظيم الكود الخاص بك وإعادة استخدامه، مما يجعل برامجك أكثر وضوحًا وفعالية وسهولة في الصيانة.

ما هي الدوال؟

الدالة هي مجموعة من التعليمات التي يتم تجميعها معًا لأداء عمل معين. يمكنك تعريفها مرة واحدة فقط، ثم استدعاؤها (تشغيلها) عدة مرات في أجزاء مختلفة من برنامجك.

مثال عملي

```
function greetUser(name) {  
  return "إني جافاسكريبت " + name + " أهلاً بك يا " ;  
}  
  
let message1 = greetUser("أحمد");  
console.log(message1);  
  
let message2 = greetUser("فاطمة");  
console.log(message2);
```

شرح الكود

في هذا المثال، قمنا بتعريف دالة اسمها `greetUser` تستقبل وسيطًا واحدًا هو `name`. وظيفتها هي بناء وإرجاع رسالة ترحيب مخصصة باستخدام الاسم المُعطى. بعد تعريف الدالة، استدعيناها مرتين بأسماء مختلفة (أحمد وفاطمة) لتوليد وطباعة رسائل ترحيب متنوعة.

- **التعريف:** تُعرّف الدوال باستخدام الكلمة المفتاحية `function` متبوعة باسم الدالة والأقواس ().
- **الوسائط:** يمكن للدوال أن تستقبل وسائط (بيانات إدخال) داخل الأقواس لتخصيص عملها.
- **الاستدعاء:** لاستدعاء الدالة وتشغيلها، تكتب اسمها متبوعًا بالأقواس ()، مع تمرير الوسائط إذا كانت موجودة.

الدوال والنطاق — تمرير الوسائط وقيم الإرجاع

تمرير الوسائط وقيم الإرجاع في جافاسكريبت

في بناء الدوال، من الشائع أن تحتاج الدالة إلى معلومات معينة لتقوم بعملها، أو أن تُخرج نتيجة بعد الانتهاء. هذه العملية تتم من خلال تمرير الوسائط (Arguments) واستقبال قيم الإرجاع (Return Values).

مفهوم تمرير الوسائط وقيم الإرجاع

الوسائط هي البيانات التي تُرسلها للدالة عند استدعائها لتستخدمها في عملياتها الداخلية. أما **قيم الإرجاع** فهي النتيجة التي تُعيدها الدالة بعد أن تُكمل مهمتها، ويمكن استخدام هذه القيمة خارج نطاق الدالة.

مثال توضيحي (Code Example)

```
function جمععدددين(num1, num2) {  
    let المجموع = num1 + num2;  
    return المجموع;  
}  
  
let نتيجة_الجمع = جمععدددين(3, 5);  
console.log("المجموع هو: 8 //", نتيجة_الجمع);
```

شرح الكود

الدالة **جمععدددين** (addNumbers) تستقبل وسيطين هما num1 و num2. داخل الدالة، يتم جمعها وتخزين النتيجة في متغير المجموع. كلمة **return** تُستخدم لإعادة قيمة هذا المتغير كناتج للدالة. عند استدعاء الدالة جمععدددين(3, 5)، يتم تمرير 3 و 5 كوسيطين، والقيمة المرجعة (8) تُخزن في المتغير نتيجة_الجمع ثم تُطبع على الشاشة.

- **الوسائط** تسمح للدوال بالعمل بمرونة على بيانات مختلفة دون الحاجة لإعادة كتابتها.
- تُستخدم كلمة المفتاح **return** لإخراج قيمة واحدة من الدالة، مما ينهي تنفيذها.
- يمكن تخزين القيمة المرجعة من الدالة في متغير، أو استخدامها مباشرة في تعبيرات أخرى.

الدوال والنطاق — نطاق المتغيرات (Scope)

نطاق المتغيرات (Scope)

يحدد نطاق المتغيرات (Scope) في جافاسكريبت الأماكن التي يمكن الوصول فيها إلى المتغيرات واستخدامها داخل برنامجك. فهم النطاق ضروري لكتابة كود منظم وخالي من الأخطاء ويساعد على تجنب التضارب بين أسماء المتغيرات.

مفهوم نطاق المتغيرات

النطاق هو "منطقة الوصول" للمتغير. عندما تعلن عن متغير، يحدد مكانه في الكود المكان الذي يمكن أن يُرى فيه هذا المتغير. هناك أنواع رئيسية: النطاق العام (Global Scope)، ونطاق الدالة (Function Scope)، ونطاق الكتلة (Block Scope) لـ `let` و `const`.

مثال برمجي

```
let رسالة_عالمية = "أهلاً من النطاق العام" // Global Scope

function عرض_الرسالة () {
  let رسالة_الدالة = "مرحباً داخل الدالة" // Function Scope
  console.log(رسالة_عالمية);
  console.log(رسالة_الدالة);
}

عرض_الرسالة(); // سيتم عرض الرسالتين

if (true) {
  let رسالة_الكتلة = "هذه رسالة داخل كتلة" // Block Scope
  console.log(رسالة_الكتلة);
}

// محاولة الوصول خارج النطاق:
// console.log(رسالة_الدالة); // سيحدث خطأ: رسالة_الدالة غير معرّفة
// console.log(رسالة_الكتلة); // سيحدث خطأ: رسالة_الكتلة غير معرّفة
```

شرح الكود

في هذا المثال، رسالة عالمية معرفة في النطاق العام، لذا يمكن الوصول إليها من أي مكان في البرنامج. أما رسالة الدالة، فقد تم تعريفها داخل الدالة عرّف الرسالة، مما يعني أنها لا يمكن الوصول إليها إلا داخل هذه الدالة فقط. وبالنسبة لرسالة الكتلة، تم تعريفها باستخدام let داخل كتلة if، وبالتالي فهي متاحة فقط ضمن تلك الكتلة. محاولة الوصول إلى رسالة الدالة أو رسالة الكتلة من خارج نطاقهما سيؤدي إلى خطأ في التنفيذ.

- **النطاق** يحدد مدى رؤية المتغيرات وإمكانية الوصول إليها في أجزاء مختلفة من الكود.
- المتغيرات المعرفة داخل **الدوال** تكون محلية لتلك الدوال ولا يمكن الوصول إليها من الخارج.
- المتغيرات المعرفة بـ let أو const داخل **الكتل البرمجية** (مثل if أو for) تكون خاصة بتلك الكتل فقط.

الدوال والنطاق — الرفع (Hoisting)

الرفع (Hoisting) في جافاسكريبت

مفهوم الرفع (Hoisting) هو سلوك فريد في جافاسكريبت يؤثر على كيفية معالجة المفسر (interpreter) لإعلانات المتغيرات والدوال. فهمه ضروري لتجنب الأخطاء وكتابة كود نظيف ومفهوم.

المفهوم الأساسي

الرفع يعني أن جافاسكريبت تقوم داخليًا بنقل إعلانات المتغيرات (المُعلنة بـ **var**) وإعلانات الدوال إلى أعلى نطاقها (scope) أثناء مرحلة الترجمة، قبل تنفيذ الكود فعليًا. هذا لا يشمل تهيئة القيم، بل فقط الإعلانات.

مثال الكود

```
console.log(greet());

function greet() {
  return "مرحباً من دالة";
}

console.log(myVar);
var myVar = "أنا متغير";
console.log(myVar);
```

شرح الكود

في المثال الأول، تم استدعاء الدالة greet() قبل تعريفها في الكود. بفضل الرفع، جافاسكريبت تعرف بوجود الدالة مسبقًا وتنفذها بنجاح، وتطبع "مرحباً من دالة!".

في المثال الثاني، عند محاولة طباعة myVar قبل الإعلان عنها، تكون القيمة undefined. هذا لأن إعلان var myVar تم رفعه، ولكن قيمة المتغير ("أنا متغير!") لم تُسند إلا في موضعها الأصلي في الكود. بعد الإسناد، يتم طباعة القيمة الصحيحة ("أنا متغير!").

- الرفع ينطبق على إعلانات الدوال والمتغيرات المُعلنة بـ **var**.
- يتم "رفع" الإعلانات (declarations) وليس التهيئة أو إسناد القيم (assignments).

- فهم الرفع يساعد في كتابة كود متوقع وتجنب السلوكيات غير المرغوبة، خاصة عند التعامل مع نطاقات المتغيرات.

الدوال والنطاق — الدوال التعبيرية ودوال رد النداء

الدوال التعبيرية ودوال رد النداء في جافاسكريبت

في هذا الدرس، سنتعرف على طريقتين مرتتين وواسعتي الاستخدام لتعريف الدوال واستدعائها في جافاسكريبت: الدوال التعبيرية التي تُعامل كقيمة، ودوال رد النداء التي تُمرر لتنفيذ لاحقًا.

الدوال التعبيرية (Function Expressions)

الدالة التعبيرية هي دالة تُعرّف كجزء من تعبير، وغالبًا ما تُسند إلى متغير. يمكن أن تكون هذه الدوال مجهولة الاسم (anonymous) أو مُسمّاة. تزيد الدوال التعبيرية من مرونة الكود لأنها تُعامل كقيم يمكن تمريرها أو إرجاعها.

دوال رد النداء (Callback Functions)

دالة رد النداء هي دالة تُمرر كحجة لدالة أخرى، وتُنفذ (يُستدعى) داخل تلك الدالة الأخرى في وقت لاحق. تُستخدم دوال رد النداء بشكل كبير في جافاسكريبت للتعامل مع الأحداث، العمليات غير المتزامنة، أو لتخصيص سلوك دالة معينة.

مثال توضيحي

```
const (الرقم_الأول, الرقم_الثاني, دالة_رد_النداء) => {  
  console.log("...جارٍ بدء العملية الحسابية");  
  دالة_رد_النداء(الرقم_الأول, الرقم_الثاني);  
  console.log("انتهت العملية الحسابية");  
};
```

```
(x, y) => {  
  const الناتج = x + y;  
  console.log(`ناتج الجمع هو: ${الناتج}`);  
};
```

```
(x, y) => {  
  const الناتج = x / y;  
  console.log(`ناتج القسمة هو: ${الناتج}`);  
};
```

شرح الكود

في هذا المثال، لدينا الدالة تنفيذ_عملية_حسابية التي تستقبل عددين ودالة أخرى تسمى دالة_رد_النداء. عندما نستدعي تنفيذ_عملية_حسابية، نمرر لها دالة تعبيرية مجهولة (مثل $(x, y) \leq \dots$) كدالة_رد_النداء. تقوم هذه الدالة الممررة بتنفيذ عملية محددة (كالجمع أو القسمة) على العددين ثم تُظهر الناتج، وتُنفذ في اللحظة المناسبة داخل الدالة الرئيسية.

- الدوال التعبيرية تُعامل كقيم ويمكن إسنادها لمتغيرات، وغالبًا ما تكون مجهولة الاسم.
- دوال رد النداء هي دوال تُمرر كحجج لدوال أخرى لتُنفذ لاحقًا.
- يُستخدمان معًا لتصميم كود مرن وقابل للتوسيع، خاصة في التعامل مع العمليات غير المتزامنة.

الكائنات الأساسية المضمنة — النصوص (Strings)

النصوص (Strings)

في لغة جافاسكريبت، النصوص (Strings) هي إحدى أنواع البيانات الأساسية التي تُستخدم لتخزين ومعالجة البيانات النصية. سواء كانت كلمة واحدة، جملة، أو فقرة كاملة، يتم التعامل معها كنص.

ما هي النصوص (Strings)؟

النص هو سلسلة من الأحرف (مثل الحروف، الأرقام، الرموز) يتم إحاطتها بعلامتي اقتباس فردية (') أو مزدوجة ("). تُعتبر النصوص كائنات مضمنة (built-in objects) في جافاسكريبت، مما يعني أنها تأتي مع خصائص (properties) ودوال (methods) جاهزة للاستخدام.

مثال على الكود

```
let firstName = "محمد";
let lastName = 'علي';
let fullName = firstName + " " + lastName;

console.log(fullName);
console.log("عدد الأحرف في الاسم الكامل:", fullName.length);
```

شرح الكود

في هذا المثال، قمنا بإنشاء متغيرين `firstName` و `lastName` لتخزين أسماء نصية. لاحظ أننا استخدمنا علامات اقتباس مزدوجة وفردية، وكلاهما صحيح. ثم قمنا بدمج هذه النصوص مع مسافة بينها لإنشاء `fullName`. أخيرًا، استخدمنا الدالة `console.log()` لعرض الاسم الكامل ثم استخدمنا الخاصية `length` لمعرفة عدد الأحرف في النص الناتج.

- النصوص هي طريقة جافاسكريبت للتعامل مع البيانات النصية.
- يجب إحاطة النصوص بعلامات اقتباس فردية (') أو مزدوجة (").
- تأتي النصوص مع خصائص ودوال مفيدة، مثل `length` لمعرفة عدد الأحرف.

الكائنات الأساسية المضمنة — طرق النصوص (String Methods)

طرق النصوص (String Methods)

تُعد طرق النصوص أدوات قوية ومدمجة في جافاسكريبت تُمكننا من التلاعب بالسلاسل النصية (Strings) وتعديلها بسهولة وكفاءة. إنها دوال جاهزة تأتي مع كل نص لمساعدتك على إنجاز مهام مثل تغيير حالة الحروف، البحث عن أجزاء معينة، أو استخراج أجزاء من النص.

ما هي طرق النصوص؟

طرق النصوص هي دوال (Functions) مُعدة مسبقاً يمكن استدعاؤها مباشرة على أي نص لتنفيذ عمليات محددة عليه. هذه العمليات تتضمن تحويل النص، البحث بداخله، قصه، أو دمج بطرق مختلفة.

مثال عملي

```
let welcomeMessage = "مرحباً بكم في جافاسكريبت";

let upperCaseMessage = welcomeMessage.toUpperCase();
let subStringExample = welcomeMessage.slice(10, 16); // "جافاس"

console.log(upperCaseMessage);
console.log(subStringExample);
```

شرح الكود

في هذا المثال، قمنا بإنشاء نص باسم `welcomeMessage`. استخدمنا دالة `toUpperCase()` لتحويل النص بأكمله إلى أحرف كبيرة، مما ينتج عنه "مَرْحَباً بِكُمْ فِي جَافَاسْكْرِيبْت!". ثم، استعملنا دالة `slice(10, 16)` لاستخراج جزء من النص، حيث يبدأ من الفهرس 10 وينتهي قبل الفهرس 16، ليُظهر لنا "جافاس".

- تُسهل طرق النصوص التعامل مع السلاسل النصية وتعديلها بشكل فعال.
- كل طريقة (دالة) تؤدي وظيفة محددة (مثل التحويل إلى أحرف كبيرة أو قص جزء).
- يمكنك دمج عدة طرق معاً لتنفيذ عمليات معقدة على النص.

الكائنات الأساسية المضمنة — الأرقام والعمليات

الرياضية (Numbers & Math)

الأرقام والعمليات الرياضية (Numbers & Math) في JavaScript

تتعامل JavaScript مع الأرقام بشكل أساسي كقيم عشرية (floating-point numbers). تشرح هذه الوحدة كيفية تعريف الأرقام وإجراء العمليات الحسابية الأساسية باستخدام الكائن Math المدمج لتنفيذ دوال رياضية أكثر تعقيدًا.

التعامل مع الأرقام والكائن Math

في JavaScript، جميع الأرقام من نوع Number، سواء كانت أعدادًا صحيحة أو عشرية. يوفر الكائن Math مجموعة من الخصائص والدوال الرياضية الثابتة التي يمكننا من إجراء عمليات مثل التقريب، الحصول على الجذر التربيعي، وتوليد الأرقام العشوائية، وغيرها.

مثال على الكود

```
let num1 = 10;
let num2 = 3.75;

let sum = num1 + num2;
let difference = num1 - num2;
let product = num1 * num2;
let quotient = num1 / num2;

let roundedNum = Math.round(num2); // يقرب لأقرب عدد صحيح
let absoluteNum = Math.abs(-5); // القيمة المطلقة
let randomNumber = Math.random(); // رقم عشوائي بين 0 و 1
```

شرح الكود

في هذا المثال، نعلن عن متغيرين رقميين num1 و num2. ثم نجري العمليات الحسابية الأساسية كالجمع والطرح والضرب والقسمة. بعد ذلك، نستخدم دوال الكائن Math: حيث Math.round() لتقريب num2، و Math.abs() للحصول على القيمة المطلقة لعدد سالب، وأخيرًا Math.random() لتوليد رقم عشوائي.

- الأرقام في JavaScript هي من نوع Number، وتشمل الأعداد الصحيحة والعشرية.

- يمكن إجراء العمليات الحسابية الأساسية (+، -، *، /) مباشرة على الأرقام.
- الكائن **Math** يوفر دوالاً رياضية متقدمة ومفيدة مثل `Math.round()` للتقريب و `Math.random()` لتوليد الأرقام العشوائية.

الكائنات الأساسية المضمنة — التواريخ والأوقات (Dates)

التواريخ والأوقات (Dates) في JavaScript

في عالم البرمجة، نحتاج غالبًا للتعامل مع التواريخ والأوقات. يوفر لنا JavaScript كائن **مضمن** وقوي يُسمى Date لإدارة وإنشاء ومعالجة معلومات التاريخ والوقت بسهولة وفعالية.

كائن Date

يسمح لنا كائن Date بتمثيل لحظة زمنية محددة. يمكنك استخدامه للحصول على التاريخ والوقت الحاليين، أو لإنشاء تاريخ ووقت معينين، ثم التلاعب بهذه القيم وعرضها بطرق مختلفة.

مثال الكود

```
// جديد يمثل الوقت الحالي Date إنشاء كائن
const now = new Date();

الحصول على مكونات التاريخ
const year = now.getFullYear();
const month = now.getMonth() + 1; // (يناير هو 0)
const day = now.getDate();

console.log(`${day}/${month}/${year}`);
console.log(`${now.toLocaleTimeString()}`);
```

شرح الكود

في هذا المثال، نقوم أولاً بإنشاء كائن Date جديد باسم now، والذي سيحتوي على التاريخ والوقت الحاليين. بعد ذلك، نستخدم دوال مثل getFullYear() و getMonth() و getDate() لاستخراج السنة والشهر واليوم. لاحظ أن getMonth() تُرجع الشهر بقيمة تبدأ من الصفر، لذا نضيف 1 للحصول على الرقم الصحيح. أخيرًا، نعرض التاريخ والوقت باستخدام القوالب النصية والدالة toLocaleTimeString() التي تعرض الوقت بتنسيق محلي.

- **الكائن Date**: هو أداة JavaScript للتعامل مع التواريخ والأوقات.
- **إنشاء التاريخ**: يمكن إنشاء كائن Date للوقت الحالي أو لتاريخ ووقت محدد.

- **استخراج المعلومات:** يوفر Date دوال لاستخراج اليوم، الشهر، السنة، الساعة، الدقيقة، وغيرها من المكونات.

الكائنات الأساسية المضمنة — تنسيق التواريخ

تنسيق التواريخ في JavaScript

تنسيق التواريخ هو جزء أساسي عند التعامل مع البيانات الزمنية، حيث يجعلها مقروءة وواضحة للمستخدمين. في JavaScript، يمكننا تحويل كائنات التاريخ (Date objects) إلى سلاسل نصية منسقة بطرق مختلفة لتلبية احتياجات العرض المتنوعة، مع الأخذ في الاعتبار اللغة والمنطقة الزمنية.

مفهوم تنسيق التواريخ

تتيح لك JavaScript عرض التواريخ والأوقات بتنسيقات متعددة. بدلاً من عرض التاريخ كـ "Mon Jul 19 2024"، يمكنك تنسيقه ليظهر كـ "الجمعة، 19 يوليو 2024" أو "14:30 2024/07/19". يتم ذلك باستخدام دوال مثل toLocaleString() التي تتيح لك تخصيص العرض بناءً على اللغة والخيارات.

مثال عملي

```
const now = new Date();

// تنسيق التاريخ والوقت باللغة العربية مع تفاصيل محددة
const formattedDate = now.toLocaleString('ar-EG', {
  weekday: 'long',
  year: 'numeric',
  month: 'long',
  day: 'numeric',
  hour: '2-digit',
  minute: '2-digit',
  second: '2-digit'
});

console.log(formattedDate);

// مثال للخروج: "الجمعة، 19 يوليو 2024 في 02:30:15 م"
```

شرح الكود

يبدأ الكود بإنشاء كائن **Date** يمثل التاريخ والوقت الحالي. ثم، يتم استدعاء الدالة **toLocaleString()** على هذا الكائن. تمرر هذه الدالة وسيطين: الأول 'ar-EG' يحدد التنسيق ليظهر باللغة العربية المصرية، والثاني هو كائن يحتوي على خيارات التنسيق مثل عرض اليوم، الشهر، السنة، والساعة والدقيقة والثانية بشكل كامل.

- **JavaScript** توفر طرقًا مرنة لتنسيق وعرض التواريخ والأوقات.
- الدالة **toLocaleString()** قوية لتخصيص عرض التاريخ والوقت بناءً على اللغة والمنطقة.
- يمكن تحديد خيارات التنسيق (مثل weekday, month, year) للتحكم الدقيق في كيفية ظهور التاريخ.

المصفوفات — تعريف المصفوفات

تعريف المصفوفات في JavaScript

مرحبًا بك في هذا الدرس! سنتعلم أساسيات تعريف المصفوفات (Arrays) في JavaScript، وهي هياكل بيانات مهمة لتخزين مجموعات من القيم بشكل منظم.

ما هي المصفوفة؟

المصفوفة هي قائمة مرتبة يمكنها تخزين عدة قيم في متغير واحد. هذه القيم يمكن أن تكون من أنواع بيانات مختلفة (أرقام، نصوص، قيم منطقية، كائنات، إلخ) ويتم الوصول إليها باستخدام الفهارس الرقمية التي تبدأ من الصفر.

مثال عملي

```
let colors = ["أحمر", "أخضر", "أزرق"];
let ages = [20, 22, 25, 30];
let mixedData = ["25", "اسم", true];

console.log(colors);
console.log(ages);
console.log(mixedData);
```

شرح الكود

في هذا المثال، قمنا بتعريف ثلاث مصفوفات باستخدام الكلمة المفتاحية `let` وعلامتي القوسين المربعين `[]`. المصفوفة `colors` تحتوي على قيم نصية، والمصفوفة `ages` تحتوي على قيم رقمية، بينما `mixedData` تحتوي على أنواع بيانات مختلفة. دالة `console.log` تستخدم لعرض محتويات كل مصفوفة كاملة في وحدة التحكم.

- المصفوفة في JavaScript هي مجموعة مرتبة من القيم المخزنة في متغير واحد.
- تُعرف المصفوفة باستخدام الأقواس المربعة `[]` ووضع العناصر بداخلها مفصولة بفاصلة.
- يمكن للمصفوفة تخزين أي نوع من البيانات، بما في ذلك أنواع مختلفة في نفس المصفوفة.

المصفوفات — الوصول للعناصر وتعديلها

الوصول للعناصر وتعديلها في المصفوفات (Arrays)

تُعد المصفوفات من هياكل البيانات الأساسية في JavaScript لتخزين مجموعة من القيم. لكي نتمكن من العمل بفعالية مع هذه المجموعات، من الضروري معرفة كيفية **الوصول إلى عناصرها الفردية** وتعديلها عند الحاجة.

الفهرسة (Index) والوصول للعناصر

يتم تخزين العناصر في المصفوفة بترتيب معين، ولكل عنصر **فهرس (Index)** فريد يبدأ من الرقم 0. للوصول إلى عنصر معين، نستخدم اسم المصفوفة متبوعًا بالأقواس المربعة [] وبداخلها رقم الفهرس الخاص بالعنصر الذي نرغب فيه.

مثال عملي

```
let fruits = ["تفاح", "موز", "برتقال"];

// الوصول للعناصر
console.log(fruits[0]); // الوصول للعنصر الأول
console.log(fruits[2]); // الوصول للعنصر الثالث

// تعديل العناصر
fruits[1] = "فراولة"; // تعديل العنصر الثاني
console.log(fruits); // عرض المصفوفة بعد التعديل

// إضافة عنصر جديد في نهاية المصفوفة (عن طريق فهرس جديد)
fruits[3] = "كيوي";
console.log(fruits);
```

شرح الكود

يبدأ المثال بتعريف مصفوفة fruits تحتوي على ثلاثة أنواع من الفاكهة. عند استخدام fruits[0]، فإننا نصل إلى العنصر الأول "تفاح". وللوصول إلى العنصر الثالث "برتقال"، نستخدم الفهرس 2. لتعديل العنصر الثاني، قمنا بتعيين قيمة جديدة "فراولة" للفهرس 1. ثم عرضنا المصفوفة لنرى التغيير. أخيرًا، أضفنا عنصرًا جديدًا "كيوي" عن طريق تعيين قيمة لفهرس غير موجود 3، مما أدى لتوسيع المصفوفة.

- يتم الوصول لعناصر المصفوفة وتعديلها باستخدام الفهرس (Index).
- يبدأ الفهرس دائمًا من **الصفر (0)** للعنصر الأول.
- يمكنك تغيير قيمة أي عنصر عن طريق تعيين قيمة جديدة لفهرسه، أو إضافة عناصر جديدة بتحديد فهرس أكبر من حجم المصفوفة الحالي.

المصفوفات — إضافة وحذف العناصر

المصفوفات في JavaScript: إضافة وحذف العناصر

تعد المصفوفات هياكل بيانات مرنة تسمح لنا بتخزين مجموعات من العناصر. في الكثير من الأحيان، نحتاج إلى تعديل محتوى هذه المصفوفات بإضافة عناصر جديدة إليها أو حذف عناصر موجودة منها. سيتناول هذا الدرس كيفية القيام بذلك في JavaScript.

مفهوم إضافة وحذف العناصر

لتعديل المصفوفات، توفر JavaScript دوالاً (methods) مدمجة يمكننا من التعامل مع عناصرها بسهولة. يمكن إضافة عناصر إلى بداية أو نهاية المصفوفة، وكذلك حذفها منها، مما يجعل المصفوفات ديناميكية للغاية.

مثال عملي

```
let colors = ["أحمر", "أخضر"];

// إضافة عنصر في نهاية المصفوفة
colors.push("أزرق");
console.log("بعد إضافة أزرق", colors);
// الناتج المتوقع: ["أحمر", "أخضر", "أزرق"]

// حذف عنصر من نهاية المصفوفة
let removedColor = colors.pop();
console.log("بعد حذف العنصر الأخير", colors);
// الناتج المتوقع: ["أحمر", "أخضر"]
console.log("العنصر المحذوف:", removedColor);
// الناتج المتوقع: "أخضر"
```

شرح الكود

في هذا المثال، استخدمنا طريقتين أساسيتين لتعديل المصفوفة:

- الطريقة **push()**: تضيف عنصرًا جديدًا إلى نهاية المصفوفة. بعد تنفيذ `colors.push("أزرق")`، أصبحت المصفوفة تحتوي على "أزرق" كعنصر أخير.

• الطريقة **pop()**: تحذف العنصر الأخير من المصفوفة وتعيده. بعد تنفيذ `colors.pop()`، تم حذف "أزرق" من المصفوفة وتخزينه في المتغير `removedColor`.
توجد أيضًا طرق مماثلة للتعامل مع بداية المصفوفة: **unshift()** لإضافة عنصر في البداية، و**shift()** لحذف عنصر من البداية.

ملخص الدرس

- تستخدم الطريقة **push()** لإضافة عنصر أو أكثر إلى نهاية المصفوفة.
- تستخدم الطريقة **pop()** لحذف العنصر الأخير من المصفوفة وإعادته.
- توجد طرق مماثلة مثل **unshift()** و **shift()** للتعامل مع بداية المصفوفة.

المصفوفات — طرق المصفوفات (Array Methods)

طرق المصفوفات (Array Methods) في JavaScript

تعلمنا سابقاً أن المصفوفات (Arrays) هي مجموعات مرتبة من البيانات. في هذا الدرس، سنتعرف على الطرق (Methods) التي توفرها JavaScript للتعامل مع المصفوفات بسهولة وفعالية، مثل الإضافة والحذف والتعديل والبحث، مما يقلل من تعقيد الكود.

ما هي طرق المصفوفات؟

طرق المصفوفات هي دوال جاهزة (built-in functions) تأتي مع كائنات المصفوفات في JavaScript. هذه الدوال تمكننا من تنفيذ عمليات شائعة ومعقدة على المصفوفات بسطر كود واحد أو أكثر، بدلاً من كتابة حلقات تكرارية طويلة يدوياً، مما يجعل الكود أنظف وأسهل للقراءة.

مثال عملي

```
const numbers = [1, 2, 3, 4, 5];

// لمضاعفة كل رقم في المصفوفة map استخدام طريقة
const doubledNumbers = numbers.map(function(number) {
    return number * 2;
});

console.log(doubledNumbers);
// الناتج: [2, 4, 6, 8, 10]
```

شرح الكود

في هذا المثال، لدينا مصفوفة تسمى **numbers** تحتوي على أرقام. استخدمنا طريقة **map()** لإنشاء مصفوفة جديدة اسمها **doubledNumbers**. تقوم دالة **map** بتطبيق دالة معينة (وهنا، مضاعفة كل رقم) على كل عنصر من عناصر المصفوفة الأصلية، ثم تُرجع مصفوفة جديدة بالنتائج دون تغيير المصفوفة الأصلية.

- طرق المصفوفات تبسط التعامل مع البيانات وتجعل الكود أكثر وضوحاً.
- توجد طرق متنوعة لأداء مهام مختلفة مثل الإضافة، الحذف، البحث، والتحويل.
- بعض الطرق (مثل map و filter) تُنشئ مصفوفة جديدة بدلاً من تعديل المصفوفة الأصلية مباشرةً.

المصفوفات — التكرار على المصفوفات (forEach, map, filter)

التكرار على المصفوفات (forEach, map, filter)

عند التعامل مع البيانات في جافاسكريبت، غالبًا ما نجد أنفسنا نعمل مع المصفوفات. التكرار على عناصر المصفوفة هو عملية أساسية لتنفيذ مهام معينة على كل عنصر، أو لتحويل المصفوفة، أو لتصفيته. سنتعلم اليوم ثلاثة طرق حديثة وفعالة لذلك.

مفاهيم أساسية للتكرار

تُعد دوال **forEach**، **map** و **filter** طرقًا قوية للتكرار على المصفوفات. تستخدم **forEach** لتنفيذ دالة معينة على كل عنصر بدون إنشاء مصفوفة جديدة. بينما **map** تنشئ مصفوفة جديدة بتطبيق دالة تحويلية على كل عنصر. أما **filter** فتنشئ مصفوفة جديدة تحتوي فقط على العناصر التي تحقق شرطًا محددًا.

مثال عملي

```
const products = [
  { name: 'Laptop', price: 1200 },
  { name: 'Mouse', price: 25 },
  { name: 'Keyboard', price: 75 },
  { name: 'Monitor', price: 300 }
];

// forEach: لعرض اسم كل منتج
products.forEach(product => {
  console.log(`المنتج: ${product.name}`);
});

// map: لإنشاء مصفوفة جديدة بأسماء المنتجات فقط
const productNames = products.map(product => product.name);
console.log("أسماء المنتجات:", productNames);

// filter: لإنشاء مصفوفة جديدة بالمنتجات الأعلى من 100 دولار
const expensiveProducts = products.filter(product => product.price > 100);
console.log("منتجات باهظة:", expensiveProducts);
```

شرح الكود

في المثال أعلاه، لدينا مصفوفة `products` تحتوي على كائنات. استخدمنا **forEach** لتصفح كل منتج وطباعة اسمه في السجل، وهذه الدالة لا تُرجع مصفوفة جديدة. ثم، استخدمنا **map** لإنشاء مصفوفة `productNames` تحتوي فقط على أسماء المنتجات بعد تطبيق دالة تحويلية. وأخيرًا، استخدمنا **filter** لإنشاء مصفوفة `expensiveProducts` تحتوي فقط على المنتجات التي يزيد سعرها عن 100 دولار، وذلك بناءً على شرط محدد.

- **forEach**: تُستخدم لتنفيذ دالة على كل عنصر في المصفوفة دون تغييرها أو إنشاء مصفوفة جديدة.
- **map**: تُستخدم لإنشاء مصفوفة جديدة بالكامل، بعد تطبيق دالة تحويلية على كل عنصر من عناصر المصفوفة الأصلية.
- **filter**: تُستخدم لإنشاء مصفوفة جديدة تحتوي على مجموعة فرعية من العناصر التي تستوفي شرطًا معينًا.

الكائنات وJSON — مقدمة إلى الكائنات

مقدمة إلى الكائنات في JavaScript

تُعد الكائنات (Objects) من المفاهيم الأساسية والقوية في JavaScript التي تساعدنا على تنظيم البيانات المعقدة بطريقة منطقية ومرتبطة، مما يجعل الكود أسهل في القراءة والإدارة.

ما هي الكائنات؟

الكائنات هي هياكل بيانات تسمح لنا بتخزين مجموعات من البيانات المرتبطة كأزواج من "المفاتيح" و"القيم". تخيلها كصناديق تحتوي على معلومات محددة عن شيء واحد، مثل شخص أو كتاب.

مثال عملي

```
const user = {
  firstName: "علي",
  lastName: "سعيد",
  age: 28,
  isVerified: true
};

console.log(user.firstName);
console.log(user.age);
```

شرح الكود

في هذا المثال، قمنا بإنشاء كائن اسمه **user**. هذا الكائن يمتلك خصائص مثل **firstName** و **lastName** و **age** و **isVerified**. كل خاصية لها اسم (مفتاح) وقيمة مرتبطة بها. تمكنا من الوصول إلى قيمة الخاصيتين **firstName** و **age** وعرضهما باستخدام اسم الكائن متبوعًا بالنقطة ثم اسم الخاصية.

- الكائنات تجمع البيانات ذات الصلة في بنية واحدة منظمة.
- تتكون الكائنات من أزواج "مفتاح-قيمة" (key-value pairs) لتخزين المعلومات.
- للوصول إلى قيمة خاصية معينة داخل كائن، نستخدم تدوين النقطة (dot notation).

الكائنات وJSON — خصائص وطرق الكائنات

خصائص وطرق الكائنات في JavaScript

في جافاسكريبت، الكائنات هي هياكل أساسية لتنظيم البيانات والوظائف المرتبطة بها. تتكون الكائنات من مكونين رئيسيين: **الخصائص** و**الطرق**. فهما ضروري لبناء تطبيقات قوية ومنظمة.

الخصائص (Properties) والطرق (Methods)

الخصائص هي البيانات أو المتغيرات المرتبطة بالكائن، مثل الاسم أو العمر. **الطرق** هي الوظائف (Functions) التي يمكن للكائن تنفيذها، وهي تمثل سلوكه أو الإجراءات التي يقوم بها.

مثال توضيحي

```
const user = {
  name: "أحمد",
  age: 25,
  greet: function() {
    return "عاماً " + this.age + " وعمرى " + this.name + " أهلاً، أنا ";
  }
};

console.log(user.name);
console.log(user.greet());
```

شرح الكود

في هذا المثال، أنشأنا كائن user لديه خاصيتان: name بقيمة "أحمد" وage بقيمة 25. كما يحتوي على طريقة greet وهي دالة تقوم بإرجاع رسالة ترحيب باستخدام الخاصيتين name وage التابعتين لنفس الكائن عبر الكلمة المفتاحية this. يتم الوصول إلى الخصائص واستدعاء الطرق باستخدام النقطة (.).

- الكائنات في جافاسكريبت تجمع البيانات (الخصائص) والسلوك (الطرق) في وحدة واحدة.
- الخصائص هي قيم تخزن معلومات عن الكائن، بينما الطرق هي دوال تنفذ إجراءات معينة.
- يمكن الوصول إلى الخصائص والطرق للكائن باستخدام علامة النقطة (.).

الكائنات وJSON — طرق الكائنات (Object Methods)

طرق الكائنات (Object Methods) في JavaScript

في JavaScript، الكائنات ليست فقط لتخزين البيانات، بل يمكنها أيضاً القيام بأفعال! طرق الكائنات هي دوال (functions) مخزنة كخصائص (properties) داخل الكائن، مما يمكنه من تنفيذ سلوكيات معينة وجعل الكائنات أكثر تفاعلية.

ما هي طرق الكائنات؟

طرق الكائنات هي دوال تابعة لكائن معين. عند استدعاء هذه الدوال، تقوم بتنفيذ مهمة أو فعل مرتبط بالكائن، وغالباً ما تستخدم خصائص الكائن نفسه لتحقيق هذه المهمة.

مثال عملي

```
const user = {
  firstName: "أحمد",
  lastName: "علي",
  age: 30,
  sayHello: function() {
    console.log(`سنة ${this.age} وعمرى ${this.lastName} ${this.firstName} أهلاً بك، أنا`);
  }
};

user.sayHello(); // استدعاء الدالة
// الناتج: أهلاً بك، أنا أحمد علي وعمرى 30 سنة
```

شرح الكود

في هذا المثال، لدينا كائن باسم user يحتوي على خصائص مثل firstName و lastName و age. الدالة sayHello هي طريقة لهذا الكائن. عندما نستدعي user.sayHello()، تقوم الدالة بطباعة رسالة ترحيب تستخدم بيانات الكائن نفسه (this.firstName و this.lastName و this.age). الكلمة المفتاحية this تشير إلى الكائن الحالي الذي يمتلك هذه الطريقة.

- **طرق الكائنات:** هي دوال تُعرف داخل الكائنات وتسمح لها بتنفيذ أفعال.

- **الوصول للخصائص:** يمكن للطريقة الوصول إلى خصائص الكائن الخاص بها باستخدام الكلمة المفتاحية `this`.
- **جعل الكائنات حيوية:** تساعد الطرق في إضافة سلوكيات ووظائف للكائنات، مما يجعلها أكثر فائدة في التطبيقات.

الكائنات وJSON — ترميز JSON

ترميز (JSON.stringify()) JSON

في هذا الدرس، سنتعلم كيفية تحويل كائنات جافاسكريبت (JavaScript Objects) إلى سلاسل نصية بصيغة JSON. هذه العملية ضرورية عند الحاجة لإرسال البيانات إلى خادم ويب، أو تخزينها، أو تبادلها مع تطبيقات أخرى.

تحويل الكائنات إلى سلاسل JSON

تتيح لنا جافاسكريبت استخدام الدالة **JSON.stringify()** لتحويل كائن جافاسكريبت عادي إلى سلسلة نصية متوافقة مع صيغة JSON. هذه السلسلة النصية هي ما يتم إرساله عبر الشبكة أو حفظه.

مثال على الكود

```
const user = {
  name: "أحمد",
  age: 30,
  city: "الرياض"
};

const jsonString = JSON.stringify(user);
console.log(jsonString);
```

شرح الكود

في هذا المثال، قمنا بإنشاء كائن جافاسكريبت يمثل معلومات مستخدم. باستخدام **JSON.stringify(user)**، تم تحويل هذا الكائن إلى سلسلة نصية بصيغة JSON. تظهر النتيجة في سجل المتصفح (console) كسلسلة JSON جاهزة للاستخدام.

- تُستخدم الدالة **JSON.stringify()** لتحويل كائن جافاسكريبت إلى سلسلة نصية بصيغة JSON.
- هذه العملية أساسية عند إرسال البيانات من المتصفح إلى الخادم أو تخزينها.
- الناتج دائمًا يكون سلسلة نصية (String)، وليس كائن جافاسكريبت.

الكائنات وJSON — تحويل البيانات إلى JSON وبالعكس

تحويل البيانات إلى JSON وبالعكس

في عالم تطوير الويب، غالبًا ما نحتاج إلى إرسال واستقبال البيانات بين تطبيقات جافاسكريبت والخوادم. تنسيق (JSON) (JavaScript Object Notation) هو الحل الأمثل لذلك، حيث يوفر طريقة قياسية وسهلة لتمثيل البيانات المنظمة. في هذا الدرس، سنتعلم كيف نحول كائنات جافاسكريبت إلى نص JSON، وكيف نعيد تحويل نص JSON إلى كائنات.

تحويل الكائنات إلى سلاسل JSON (JSON.stringify())

تستخدم دالة JSON.stringify() لتحويل كائن جافاسكريبت إلى سلسلة نصية بتنسيق JSON. هذه السلسلة النصية هي ما يتم إرساله عادةً عبر الشبكة أو تخزينه.

تحويل سلاسل JSON إلى كائنات (JSON.parse())

تستخدم دالة JSON.parse() لتحويل سلسلة نصية بتنسيق JSON إلى كائن جافاسكريبت يمكن التعامل معه واستخدام بياناته داخل التطبيق.

مثال عملي

```
const userProfile = {
  name: "ليلى أحمد",
  age: 28,
  isStudent: false,
  hobbies: ["قراءة", "سفر"]
};

// JSON تحويل الكائن إلى نص
const jsonString = JSON.stringify(userProfile);
console.log("JSON String:", jsonString);

// إلى كائن جافاسكريبت JSON تحويل نص
const parsedObject = JSON.parse(jsonString);
console.log("Parsed Object:", parsedObject);
```

```
console.log("Name from parsed object:", parsedObject.name);
```

شرح الكود

بدأنا بتعريف كائن جافاسكريبت باسم `userProfile` يحتوي على معلومات مختلفة. ثم استخدمنا `JSON.stringify()` لتحويل هذا الكائن إلى سلسلة نصية بتنسيق `JSON`، والتي تم طباعتها على الشاشة. لاحقًا، أخذنا نفس السلسلة النصية `JSON` واستخدمنا `JSON.parse()` لإعادة تحويلها إلى كائن جافاسكريبت يمكن الوصول إلى خصائصه، مثل `parsedObject.name`.

- **`JSON.stringify()`**: يحول كائن جافاسكريبت إلى سلسلة نصية بتنسيق `JSON`.
- **`JSON.parse()`**: يحول سلسلة نصية بتنسيق `JSON` إلى كائن جافاسكريبت.
- هذه الوظائف ضرورية لتبادل البيانات بين تطبيقك وال خادم أو تخزينها.

مميزات ES6 الحديثة — تعريف المتغيرات (let, const)

تعريف المتغيرات (let, const)

في JavaScript، قدمت مميزات ES6 طرقًا جديدة وأفضل لتعريف المتغيرات باستخدام **let** و **const**، والتي تحل العديد من المشكلات المتعلقة بـ **var** القديمة وتوفر تحكمًا أكبر في نطاق المتغيرات.

لماذا let و const؟

تسمح لك **let** بتعريف متغيرات يمكن إعادة تعيين قيمتها لاحقًا، لكنها تلتزم بالنطاق الكتلي (Block Scope). بينما تُستخدم **const** لتعريف ثوابت لا يمكن إعادة تعيين قيمتها بعد تعريفها الأولي، وهي أيضًا تلتزم بالنطاق الكتلي. هذا يوفر كودًا أكثر وضوحًا وأقل عرضة للأخطاء.

مثال عملي

```
function greetUser(name) {  
  let message = "مرحباً بك";  
  if (name === "أحمد") {  
    let specialMessage = "أهلاً بك يا أحمد";  
    console.log(specialMessage);  
  }  
  const API_KEY = "xyz123";  
  // API_KEY = "new_key"; // السطر سيسبب خطأ  
  console.log(message);  
  console.log("المفتاح السري", API_KEY);  
}  
  
greetUser("أحمد");
```

شرح الكود

في هذا المثال، قمنا بتعريف `message` باستخدام **let**، مما يجعلها متاحة داخل الدالة ويمكن تغيير قيمتها. المتغير `specialMessage` معرف أيضًا بـ **let** ولكن داخل كتلة `if`، مما يعني أنه لا يمكن الوصول إليه خارج تلك الكتلة. أما `API_KEY` فهو ثابت تم تعريفه بـ **const**، ولا يمكن إعادة تعيين قيمته لاحقًا، محاولتك لذلك ستؤدي إلى خطأ برمجي.

- استخدم **let** لتعريف المتغيرات التي قد تتغير قيمتها لاحقًا، وتذكر أنها تلتزم بالنطاق الكتلي.
- استخدم **const** لتعريف الثوابت التي يجب ألا تتغير قيمتها أبدًا بعد تعريفها الأولي، وهي أيضًا تلتزم بالنطاق الكتلي.
- تجنب استخدام **var** في مشاريعك الحديثة للحصول على كود أكثر تنظيمًا وأمانًا.

مميزات ES6 الحديثة — دوال السهم (Arrow Functions)

مميزات ES6 الحديثة: دوال السهم (Arrow Functions)

دوال السهم هي إضافة قوية في ES6 (ECMAScript 2015) غيّرت طريقة كتابة الدوال في JavaScript. تساعد هذه الدوال في كتابة كود أقصر وأكثر وضوحاً، خاصة عند التعامل مع الدوال المجهولة (anonymous functions)، وتُقدم حلاً أنيقاً لمشكلة سياق this الشائعة.

المفهوم الأساسي: تعريف دوال السهم

تقدم دوال السهم طريقة مختصرة لكتابة الدوال في JavaScript، وتتميز ببناء جملة أقل تعقيداً. تُعرف غالباً باستخدام الرمز =>، وتُسهّل كتابة الدوال السطرية القصيرة (inline functions) والدوال المجهولة.

مثال الكود

```
// Traditional Function
const greetTraditional = function(name) {
  return "مرحباً يا " + name + "!";
};
console.log(greetTraditional("علي"));

// Arrow Function (single line)
const greetArrow = (name) => "مرحباً يا " + name + "!";
console.log(greetArrow("فاطمة"));

// Multi-line Arrow Function
const calculateSum = (a, b) => {
  let sum = a + b;
  return sum;
};
console.log(calculateSum(10, 20));
```

شرح الكود

في هذا المثال، نرى مقارنة بين طريقة تعريف دالة تقليدية ودالة سهم. الدالة greetTraditional هي دالة عادية، بينما greetArrow هي دالة سهم تقوم بنفس المهمة ولكن ببناء جملة أقصر وأكثر إيجازاً. الدالة calculateSum توضح كيفية استخدام دوال السهم لأكثر من سطر من التعليمات، حيث نستخدم أقواس متعرجة {} وكلمة return بشكل صريح.

نقاط رئيسية

- **بناء جملة أقصر:** تُمكنك من كتابة الدوال بطريقة مختصرة، خاصة للدوال البسيطة ذات السطر الواحد.
- **تبسيط this:** تحل مشكلة سياق (context) this في JavaScript، حيث تحتفظ بـ this من السياق المحيط بها.
- **مثالية للدوال المجهولة:** مفيدة جداً عند تمرير دوال كمعاملات لدوال أخرى (مثل map، filter، reduce، أو عند استخدام الـ callbacks).

مميزات ES6 الحديثة — القوالب النصية (Template Literals)

القوالب النصية (Template Literals)

تُعد القوالب النصية إحدى الميزات القوية التي قُدمت في ES6 (ECMAScript 2015) في JavaScript، وتهدف إلى تبسيط التعامل مع السلاسل النصية (strings)، خاصة عند الحاجة إلى دمج المتغيرات أو كتابة نصوص متعددة الأسطر.

المفهوم الأساسي

تسمح القوالب النصية بإنشاء سلاسل نصية باستخدام علامتي الاقتباس الخلفيتين (``) بدلاً من الاقتباسات المفردة أو المزدوجة. الميزة الرئيسية هي إمكانية تضمين المتغيرات والتعبيرات مباشرة داخل النص باستخدام التركيب `{expression}`.

مثال برمجي

```
const اسم_المستخدم = "سارة";
const العمر = 28;
.التحية = `أهلاً، اسمي ${اسم_المستخدم} وعجري ${العمر} عام
const console.log(التيحية);
```

شرح الكود

في هذا المثال، قمنا بإنشاء متغيرين: `اسم_المستخدم` و `العمر`. ثم استخدمنا القوالب النصية (بواسطة علامتي الاقتباس الخلفيتين) لإنشاء سلسلة نصية `.التحية`. لاحظ كيف تم إدراج قيمة `اسم_المستخدم` و `العمر` مباشرة داخل السلسلة باستخدام `${اسم_المستخدم}` و `${العمر}`، مما يجعل الكود أكثر وضوحاً وسهولة في القراءة مقارنةً بطريقة الدمج التقليدية. كما يدعم المثال كتابة نص متعدد الأسطر مباشرة.

- تستخدم القوالب النصية علامتي الاقتباس الخلفيتين (``) لتعريف السلاسل النصية.
- تسمح بتضمين المتغيرات والتعبيرات داخل السلسلة باستخدام `${}`.
- تبسط كتابة السلاسل النصية متعددة الأسطر دون الحاجة لأحرف خاصة (مثل `\n`).

مميزات ES6 الحديثة — تفكيك المهيكلية (Destructuring)

تفكيك المهيكلية (Destructuring)

تفكيك المهيكلية هي ميزة قوية أُضيفت في ES6، تسمح لك باستخراج القيم من المصفوفات أو الخصائص من الكائنات وتعيينها لمتغيرات جديدة بطريقة أكثر سهولة ووضوحًا. إنها تجعل الكود الخاص بك أقصر وأسهل في القراءة.

ما هو تفكيك المهيكلية؟

تتيح لك هذه الميزة "تفكيك" البيانات الموجودة داخل الكائنات أو المصفوفات وتعيينها لمتغيرات جديدة ببناء جملة موجز. يمكن استخدامها لاستخراج قيم فردية أو حتى أجزاء فرعية من هياكل البيانات المعقدة.

مثال عملي

```
const user = {
  id: 1,
  name: 'أحمد',
  email: 'ahmed@example.com'
};

// تفكيك الكائن
const { name, email } = user;
console.log(name); // أحمد
console.log(email); // ahmed@example.com

const fruits = ['تفاح', 'برتقال', 'موز'];

// تفكيك المصفوفة
const [firstFruit, secondFruit] = fruits;
console.log(firstFruit); // تفاح
console.log(secondFruit); // برتقال
```

شرح الكود

في الجزء الأول من المثال، قمنا بإنشاء كائن user. باستخدام تفكيك المهيكلة، استخرجنا قيمتي name و email مباشرة من الكائن وقمنا بتعيينهما لمتغيرات جديدة تحمل نفس الأسماء. في الجزء الثاني، لدينا مصفوفة fruits. وبنفس الطريقة، قمنا بتفكيك أول عنصرين في المصفوفة وتعيينهما لمتغيري firstFruit و secondFruit.

- **تبسيط الوصول:** تجعل الوصول إلى عناصر الكائنات والمصفوفات أكثر سهولة واختصارًا.
- **كود أنظف:** تقلل من الحاجة لكتابة كود متكرر لاستخراج البيانات، مما يجعل الكود أكثر قابلية للقراءة.
- **مرونة:** يمكن استخدامها مع القيم الافتراضية، وتسمية المتغيرات المختلفة، وتفكيك الكائنات المتداخلة أو المصفوفات.

مميزات ES6 الحديثة — الفئات (Classes)

الفئات (Classes)

في JavaScript ES6، قُدمت **الفئات (Classes)** لتبسيط عملية إنشاء الكائنات وتطبيق مفاهيم البرمجة كائنية التوجه (OOP) بطريقة أكثر وضوحًا وتنظيمًا. هي في الأساس "سكر نحوي" (syntactic sugar) فوق النماذج الأولية (prototypes) الموجودة في JavaScript.

ما هي الفئات؟

الفئات هي قوالب (blueprints) تُستخدم لإنشاء كائنات (Objects) لها خصائص (Properties) وأساليب (Methods) مشتركة. تسمح لك بتعريف هيكل ثابت للكائنات، مما يسهل إعادة استخدام الكود وإدارته.

مثال عملي

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHello() {
    return `سنة ${this.age} وعمري ${this.name} مرحباً، اسمي`
  }
}

const user = new Person('أحمد', 30);
console.log(user.sayHello());
```

شرح الكود

في هذا المثال، قمنا بإنشاء فئة اسمها Person. تحتوي على دالة بناء (constructor) تستقبل name و age لتهيئة الكائن عند إنشائه. كما تحتوي على دالة sayHello التي تُرجع رسالة تحية. بعد ذلك، أنشأنا كائنًا جديدًا user من الفئة Person وقمنا باستدعاء دالة sayHello الخاصة به.

- تُبسّط الفئات عملية بناء الكائنات المعقدة وتجعل الكود أكثر قابلية للقراءة.
- تُستخدم دالة constructor لتهيئة خصائص الكائنات الجديدة عند إنشائها.
- تُساعد الفئات في تنظيم الكود وتطبيقه بمفاهيم البرمجة كائنية التوجه بشكل أنيق.

مميزات ES6 الحديثة — المعاملات المنتشرة والمتبقية

(Spread and Rest Operators)

المعاملات المنتشرة والمتبقية (Spread and Rest Operators)

تُعد المعاملات المنتشرة والمتبقية (...) من الإضافات القوية في ES6 (ECMAScript 2015) التي تُبسّط التعامل مع المصفوفات، الكائنات، وقوائم الوسائط في الدوال. على الرغم من أنهما يستخدمان نفس الرمز، إلا أن وظيفتهما تختلف بناءً على السياق الذي يظهران فيه.

معامل الانتشار (...) Spread Operator

يسمح لك معامل الانتشار بتوسيع (نشر) عناصر مصفوفة أو سلسلة أو كائن قابل للتكرار في أماكن يتم فيها توقع عناصر متعددة، مثل عند دمج مصفوفتين أو نسخ كائن.

المعامل المتبقي (...) Rest Parameter

يستخدم المعامل المتبقي في تعريف الدوال لجمع عدد غير محدد من الوسائط (التي لم يتم تحديدها بوسائط أخرى) في مصفوفة واحدة. هذا يسهل التعامل مع الدوال التي تقبل عددًا متغيرًا من المدخلات.

مثال عملي

```
// استخدام معامل الانتشار (...) لدمج ونسخ 1.
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4]; // [4, 3, 2, 1] دمج:

const person = { name: 'أحمد', age: 30 };
const updatedPerson = { ...person, city: 'جدة' }; // نسخ وإضافة: { name: 'أحمد', age: 30, city: 'جدة' }

// استخدام المعامل المتبقي (...) في الدوال 2.
function calculateAverage(name, ...grades) { // 'grades' الأرقام
  ستكون مصفوفة تحتوي على جميع
  'name' بعد
  const sum = grades.reduce((acc, grade) => acc + grade, 0);
  return `${name}'s average: ${sum / grades.length}`;
}
```

```
console.log(calculateAverage('92,78,90,85','فاطمة'));  
// الناتج: فاطمة 86.25
```

شرح الكود

في الجزء الأول، استخدمنا معامل الانتشار ... **لدمج مصفوفتين** في arr2 و **نسخ كائن** person وإضافة خاصية جديدة إليه لإنشاء updatedPerson. يقوم المعامل بنشر عناصر الكيان الأصلي. أما في الدالة calculateAverage، فقد استخدمنا المعامل المتبقي grades... لجمع جميع الأرقام التي تم تمريرها بعد الوسيط الأول (name) في مصفوفة واحدة تسمى grades، مما يسمح بحساب متوسط أي عدد من الدرجات.

- معامل الانتشار (...) يسمح بنشر عناصر مصفوفة أو كائن في سياق آخر.
- المعامل المتبقي (...) يجمع الوسائط المتبقية للدالة في مصفوفة واحدة.
- كلاهما يعززان مرونة ووضوح الشفرة عند التعامل مع البيانات الديناميكية ووسائط الدوال.

التفاعل مع صفحة الويب (DOM) — مقدمة إلى DOM

مقدمة إلى DOM

في تطوير الويب، تُعد القدرة على التفاعل مع محتوى وتصميم صفحة الويب أمراً أساسياً لبناء تطبيقات ديناميكية. هذا التفاعل ممكن بفضل (Document Object Model) **DOM**، وهو ما سنتعرف عليه في هذا الدرس.

ما هو DOM؟

DOM هو واجهة برمجة تطبيقات (API) للمستندات HTML و XML. ببساطة، يمثل DOM صفحة الويب كـ "شجرة" من الكائنات (Nodes). كل عنصر في صفحة HTML (مثل الفقرات، الصور، الأزرار) يصبح عقدة (Node) في هذه الشجرة. يسمح لك هذا التمثيل الشجري بالوصول إلى العناصر وتغييرها أو حذفها أو إضافة عناصر جديدة باستخدام JavaScript.

مثال عملي

```
<-- HTML في ملف -->
<p id="myParagraph">مرحباً بالعالم</p>

// JavaScript في ملف
const paragraphElement = document.getElementById('myParagraph');
paragraphElement.textContent = 'لقد تغير النص باستخدام JavaScript!';
paragraphElement.style.color = 'blue';
```

شرح الكود

يستخدم الكود في JavaScript الأمر `document.getElementById('myParagraph')` للوصول إلى عنصر الفقرة الذي يحمل المعرف `myParagraph` من صفحة HTML. بعد الحصول على العنصر، يقوم بتغيير النص الداخلي له باستخدام خاصية `textContent`، ثم يغير لون النص إلى الأزرق باستخدام خاصية `style.color`. هذا يوضح كيف يمكن للجافاسكريبت التفاعل مع محتوى وتصميم عناصر DOM.

- **DOM** هو تمثيل شجري لصفحة الويب يسمح للجافاسكريبت بالتفاعل معها.
- يمكنك استخدام **DOM** للوصول إلى عناصر HTML وتعديل محتواها أو خصائصها.
- يُمكن **DOM** مطوري الويب من إنشاء صفحات ديناميكية ومتفاعلة.

التفاعل مع صفحة الويب (DOM) — الوصول للعناصر (Selecting Elements)

الوصول للعناصر (Selecting Elements)

للتفاعل مع صفحة الويب وتغيير محتواها أو مظهرها باستخدام JavaScript، نحتاج أولاً إلى "الوصول" لعناصر HTML الموجودة فيها. هذه العملية أساسية وتسمح لنا باستهداف أجزاء محددة من الصفحة للتعامل معها.

كيف نختار العناصر؟

توفر واجهة برمجة تطبيقات المستند (DOM) عدة طرق لاختيار العناصر بناءً على خصائصها مثل الـ **ID** الفريد، أو الـ **class** المشترك، أو اسم الوسم (tag name)، أو باستخدام المحددات (selectors) الشبيهة بـ CSS.

مثال الكود

```
// الوصول لعنصر باستخدام ID
const mainTitle = document.getElementById('pageTitle');
console.log(mainTitle.textContent);

// الوصول لعناصر باستخدام Class Name
const introParagraphs = document.getElementsByClassName('intro-text');
console.log(introParagraphs[0].textContent);

// الوصول لأول عنصر يطابق محدد CSS
const firstParagraph = document.querySelector('p');
console.log(firstParagraph.textContent);
```

شرح الكود

في الكود أعلاه، استخدمنا طرقًا مختلفة للوصول للعناصر. `document.getElementById()` يختار عنصرًا واحدًا وفريدًا بناءً على معرفه (ID). `document.getElementsByClassName()` يعيد مجموعة من العناصر التي تشترك في نفس الفئة (class). أما `document.querySelector()` فيختار أول عنصر يطابق محدد CSS الذي نمرره له، مما يجعله مرئيًا جدًا.

نقاط أساسية

- `getElementById()`: مثالي لاختيار عنصر HTML واحد وفريد.
- `getElementsByClassName()`: يستخدم لاختيار مجموعة من العناصر التي تشترك في نفس الفئة.
- `querySelector()`: طريقة حديثة ومرنة لاختيار أول عنصر يطابق أي محدد CSS.

التفاعل مع صفحة الويب (DOM) — تعديل العناصر

(Modifying Elements)

تعديل العناصر في DOM

بعد أن تعلمنا كيفية تحديد العناصر في صفحة الويب باستخدام JavaScript وواجهة DOM، حان الوقت لتتعلم كيف يمكننا تغيير هذه العناصر وتحديثها ديناميكيًا. يتيح لنا DOM تعديل محتوى العناصر، خصائصها، وأنماطها (CSS)، مما يجعل صفحات الويب أكثر تفاعلية وحيوية.

مفهوم تعديل العناصر

تعديل العناصر يعني تغيير مظهرها أو محتواها بعد تحميل الصفحة. يمكننا تغيير النص داخل العنصر، إضافة أو إزالة فئات CSS، تعديل سمات HTML مثل src للصور أو href للروابط، أو حتى تغيير أنماط CSS المباشرة للعنصر.

مثال على الكود

```
const messageElement = document.getElementById('greetingMessage');

// تغيير المحتوى النصي للعنصر
messageElement.textContent = 'لقد تم تعديل الرسالة بواسطة JavaScript!';

// تغيير لون النص وحجم الخط
messageElement.style.color = 'green';
messageElement.style.fontSize = '20px';
```

شرح الكود

في هذا المثال، قمنا أولاً بالحصول على العنصر الذي يحمل المعرف greetingMessage من صفحة الويب. ثم قمنا بتغيير النص الظاهر داخل هذا العنصر باستخدام خاصية textContent. بعد ذلك، استخدمنا خاصية style لتعديل اللون وحجم الخط الخاصين بالنص، حيث غيرنا اللون إلى الأخضر وحجم الخط إلى 20 بكسل.

- **التحكم الكامل:** DOM يتيح لك تغيير أي جزء من العنصر المرئي أو غير المرئي.
- **textContent للمحتوى النصي:** استخدم هذه الخاصية لتعديل أو قراءة النص داخل العنصر.

- **style** **للأنماط:** استخدم هذه الخاصية للوصول إلى خصائص CSS وتعديلها مباشرة (مثل style.color أو style.backgroundColor).

التفاعل مع صفحة الويب (DOM) — إضافة وحذف العناصر

التفاعل مع صفحة الويب (DOM): إضافة وحذف العناصر

تُعد القدرة على **إضافة وحذف العناصر** ديناميكيًا من صفحة الويب باستخدام JavaScript وواجهة برمجة تطبيقات المستند (DOM) مهارة أساسية. تسمح لك هذه الميزة ببناء صفحات ويب تفاعلية تتغير محتواها استجابةً لأفعال المستخدم أو البيانات الجديدة، مما يعزز تجربة المستخدم بشكل كبير.

مفهوم إضافة وحذف العناصر

لإضافة عنصر جديد إلى الصفحة، نقوم أولاً بإنشائه في الذاكرة باستخدام `document.createElement()`، ثم نحدد خصائصه ومحتواه، وأخيرًا نضيفه إلى عنصر أب موجود في الصفحة باستخدام `parentNode.appendChild()`. لحذف عنصر، نحدد العنصر المراد حذفه ثم نستدعي الدالة `remove()` عليه مباشرةً.

مثال الكود

```
const newParagraph = document.createElement('p');
newParagraph.textContent = 'هذه فقرة جديدة أضيفت بواسطة JavaScript!';
newParagraph.style.color = 'green';
document.body.appendChild(newParagraph);

const deleteButton = document.createElement('button');
deleteButton.textContent = 'احذف هذا الزر';
document.body.appendChild(deleteButton);

deleteButton.addEventListener('click', () => {
    deleteButton.remove();
});
```

شرح الكود

يقوم الكود بإنشاء عنصر `<p>` جديد، ويُعيّن محتواه النصي ولونه، ثم يضيفه إلى `<body>` الصفحة. بعد ذلك، ينشئ زرًا ويضيفه إلى `<body>` أيضًا. عند النقر على هذا الزر، يستدعي الكود دالة `remove()` ليقوم الزر بحذف

نفسه من الصفحة.

- **إنشاء العناصر:** استخدم `document.createElement('tagName')` لإنشاء أي عنصر HTML جديد في الذاكرة.
- **إضافة العناصر:** استخدم `parentNode.appendChild(childElement)` لإضافة عنصر كابن لعنصر موجود في الصفحة.
- **حذف العناصر:** استخدم `element.remove()` لإزالة عنصر محدد مباشرةً من بنية DOM.

التفاعل مع صفحة الويب (DOM) — الأحداث (Events)

الأحداث (Events)

الأحداث هي أساس التفاعل الديناميكي في صفحات الويب. تخيل أن صفحتك يمكنها "الاستماع" لأفعال المستخدم، مثل النقر على زر أو إدخال نص. الأحداث تُمكن JavaScript من الاستجابة لهذه الأفعال، مما يجعل تجربتك مع الموقع حيوية ومتجاوبة بشكل فوري.

معالجة الأحداث (Event Handling)

لجعل صفحة الويب تتفاعل، نستخدم وظائف خاصة تُسمى "معالجات الأحداث" (Event Handlers). هذه الوظائف يتم تشغيلها تلقائيًا عندما يحدث حدث معين على عنصر HTML. الطريقة الأكثر شيوعًا وفعالية للقيام بذلك هي استخدام `addEventListener()`.

مثال عملي

```
const myButton = document.getElementById('actionButton');
const displayArea = document.getElementById('displayMessage');

// إضافة مستمع لحدث "النقر" على الزر
myButton.addEventListener('click', function() {
  displayArea.textContent = 'أهلاً بك! تم التفاعل';
  displayArea.style.backgroundColor = 'yellow';
});
```

شرح الكود

في هذا المثال، قمنا أولاً بالحصول على عنصر الزر والفقرة المخصصة للعرض باستخدام `document.getElementById()`. ثم استخدمنا `addEventListener()` على الزر، مع تحديد حدث `'click'` (أي عند النقر). عند وقوع هذا الحدث، يتم تنفيذ الدالة المجهولة التي تقوم بتغيير نص الفقرة ولونها، مما يوضح الاستجابة للتفاعل.

- **الأحداث** تجعل صفحات الويب تفاعلية وتستجيب لأفعال المستخدمين.
- `addEventListener()` هي الطريقة القياسية والفعالة للاستماع للأحداث ومعالجتها.
- تشمل الأحداث الشائعة: `click` (نقر)، `submit` (إرسال نموذج)، `mouseover` (تمرير الفأرة)، و `keydown` (الضغط على مفتاح).

التفاعل مع صفحة الويب (DOM) — مستمعو الأحداث (Event Listeners)

مستمعو الأحداث (Event Listeners)

في تفاعلك مع صفحة الويب، غالبًا ما تحتاج إلى جعل موقعك يستجيب لأفعال المستخدم، مثل النقر على زر أو تحريك الماوس. هنا يأتي دور **مستمعو الأحداث** في JavaScript، فهي تمكننا من "الاستماع" لأحداث معينة تحدث على عناصر HTML وتنفيذ كود معين عند وقوع هذه الأحداث.

المفهوم الأساسي

مستمع الحدث هو وظيفة تنتظر وقوع حدث محدد (مثل نقرة أو إدخال) على عنصر معين في صفحة الويب. عندما يقع هذا الحدث، يتم تشغيل الكود المرتبط بمستمع الحدث تلقائيًا، مما يجعل صفحتك ديناميكية وتفاعلية.

مثال الكود

```
// HTML for context (not part of JS code block): <button id="myButton">اضغط هنا</button>
```

```
const myButton = document.getElementById('myButton');
```

```
myButton.addEventListener('click', function() {
```

```
    alert('لقد ضغطت على الزر');
```

```
    myButton.textContent = 'تم الضغط';
```

```
});
```

شرح الكود

أولاً، نحدد الزر في صفحة الويب باستخدام معرفه (myButton). ثم نستخدم الدالة addEventListener() لربط مستمع حدث بهذا الزر. الحدث الذي نستمع إليه هنا هو 'click' (النقر). الوظيفة التي تمرر كمعامل ثاني هي الكود الذي سيتم تشغيله عندما ينقر المستخدم على الزر. في هذا المثال، تظهر رسالة تنبيه ثم يتغير نص الزر إلى "تم الضغط!".

- **التفاعل الديناميكي:** يمكنك مستمعو الأحداث من جعل صفحات الويب تتفاعل مع المستخدمين بشكل حيوي.
- **ربط الوظائف بالأحداث:** تستخدم addEventListener () لربط وظيفة معينة بحدث يقع على عنصر HTML.
- **أحداث متعددة:** يمكنك الاستماع لأنواع مختلفة من الأحداث، مثل النقر (click)، وتغيير القيمة (change)، وتحريك الماوس (mouseover)، وغيرها.

البرمجة غير المتزامنة — مقدمة إلى عدم التزامن

مقدمة إلى عدم التزامن في JavaScript

تستكشف هذه المقدمة مفهوم البرمجة غير المتزامنة في JavaScript، وهو أمر حيوي لبناء تطبيقات ويب سريعة الاستجابة. سنتعلم كيف يمكن لـ JavaScript التعامل مع المهام الطويلة الأمد دون تجميد واجهة المستخدم، مما يحسن تجربة المستخدم بشكل كبير.

ما هي البرمجة غير المتزامنة؟

البرمجة غير المتزامنة تسمح لـ JavaScript بتنفيذ مهام متعددة "في الخلفية" دون حجب أو إيقاف التنفيذ الرئيسي للبرنامج. هذا يعني أن المتصفح يمكنه الاستمرار في الاستجابة لتفاعلات المستخدم بينما يتم جلب البيانات من خادم أو إجراء عمليات تستغرق وقتاً طويلاً.

مثال كود

```
console.log("الخطوة 1: البدء");

setTimeout(() => {
  console.log("الخطوة 2: هذه الرسالة تظهر بعد تأخير (غير متزامن)");
  // تأخير لمدة 2000 ميلي ثانية (ثانيتين)
}, 2000);

console.log("الخطوة 3: التنفيذ مستمر (قبل الرسالة المتأخرة)");
```

شرح الكود

في هذا المثال، يتم طباعة "الخطوة 1" و "الخطوة 3" فوراً لأنها مهام متزامنة. أما الدالة `setTimeout` فتأخذ دالة أخرى وتؤخر تنفيذها لمدة ثانيتين دون إيقاف باقي الكود. لذلك، سترى "الخطوة 3" قبل "الخطوة 2"، مما يوضح كيف تستمر JavaScript في العمل أثناء انتظار مهمة غير متزامنة لإكمالها.

- **منع التجمد:** البرمجة غير المتزامنة تمنع تطبيقات JavaScript من "التجمد" أثناء المهام الطويلة.
- **أدوات عدم التزامن:** `setTimeout` هو مثال بسيط على كيفية جدولة المهام لتنفيذها لاحقاً دون حجب.
- **تجربة مستخدم أفضل:** عدم التزامن يسمح بتجربة مستخدم أكثر سلاسة وتطبيقات أكثر استجابة.

البرمجة غير المتزامنة — دوال التأخير الزمني (setTimeout, setInterval)

دوال التأخير الزمني (setTimeout, setInterval)

في البرمجة غير المتزامنة، نحتاج أحيانًا لتأخير تنفيذ كود معين أو لتكراره بانتظام. توفر JavaScript دوالاً قوية مثل `setTimeout` و `setInterval` لإدارة هذه المهام بفعالية دون حظر تنفيذ باقي البرنامج.

التحكم في توقيت التنفيذ

تسمح لك هذه الدوال بجدولة تنفيذ وظيفة (دالة) بعد فترة زمنية محددة. تستخدم `setTimeout` لتنفيذ الدالة مرة واحدة فقط بعد التأخير المحدد، بينما تستخدم `setInterval` لتنفيذ الدالة بشكل متكرر كل فترة زمنية معينة.

مثال عملي

```
console.log("...بدء البرنامج");

setTimeout(() => {
  console.log("أهلاً بك بعد 2 ثانية");
}, 2000); // 2000 ميلي ثانية = 2 ثانية

let counter = 0;
const intervalRef = setInterval(() => {
  counter++;
  console.log(`العداد: ${counter}`);
  if (counter === 3) {
    clearInterval(intervalRef); // إيقاف العداد
    console.log("تم إيقاف العداد");
  }
}, 1000); // 1000 ميلي ثانية = 1 ثانية

console.log("...جاري تنفيذ باقي الكود");
```

شرح الكود

عند تشغيل هذا الكود، ستظهر رسالة "بدء البرنامج..." ثم "جاري تنفيذ باقي الكود..." فوراً، لأن دوال التأخير لا تحظر التنفيذ. بعد ثانيتين، ستظهر رسالة "أهلاً بك بعد 2 ثانية!" التي تم تحديدها بواسطة `setTimeout`. أما `setInterval`، فستقوم بزيادة عداد وطباعته كل ثانية لمدة ثلاث مرات، ثم تتوقف الدالة المتكررة باستخدام `clearInterval`.

- **setTimeout**: لتنفيذ دالة مرة واحدة بعد فترة تأخير محددة.
- **setInterval**: لتنفيذ دالة بشكل متكرر كل فترة زمنية معينة.
- كلا الدالتين **غير متزامنتين**، مما يعني أنهما لا تعيقان تنفيذ باقي الكود.
- يجب استخدام **clearInterval** لإيقاف `setInterval` ومنعها من التكرار اللانهائي.

البرمجة غير المتزامنة — الوعود (Promises)

الواعد (Promises) في جافاسكريبت

في البرمجة غير المتزامنة، نحتاج أحيانًا للتعامل مع عمليات تستغرق وقتًا، مثل جلب بيانات من الإنترنت أو قراءة ملف. الوعود في جافاسكريبت توفر طريقة منظمة وأنيقة للتعامل مع النتائج المستقبلية لهذه العمليات، سواء كانت ناجحة أو فاشلة.

ما هي الوعود؟

الوعد (Promise) هو كائن في جافاسكريبت يمثل قيمة قد تكون متاحة الآن، أو في المستقبل، أو لا تكون متاحة على الإطلاق. إنه بمثابة "تعهد" بأن نتيجة عملية غير متزامنة ستأتي في وقت ما، إما بنجاح أو بفشل.

مثال برمجي

```
const جلب_البيانات_من_الخادم = new Promise((resolve, reject) => {
    setTimeout(() => {
        const نجاح_العملية = true; // محاكاة لنجاح أو فشل عملية الخادم
        if (نجاح_العملية) {
            resolve("اتم استلام البيانات بنجاح");
        } else {
            reject("فشل في جلب البيانات من الخادم");
        }
    }, 2000);
});

جلب_البيانات_من_الخادم
    .then((رسالة_نجاح) => {
        console.log(رسالة_نجاح);
    })
    .catch((رسالة_خطأ) => {
        console.error(رسالة_خطأ);
    });
```

شرح الكود

يبدأ الكود بإنشاء وعد جديد يسمى جلب البيانات من الخادم. داخل الوعد، استخدمنا `setTimeout` لمحاكاة عملية تستغرق ثانيتين. إذا كانت نجاح العملية صحيحة، يتم استدعاء `resolve` مع رسالة النجاح، مما يعني أن الوعد قد تم بنجاح. وإلا، يتم استدعاء `reject` مع رسالة خطأ. بعد ذلك، نستخدم `.then()` للتعامل مع حالة النجاح و `.catch()` للتعامل مع حالة الفشل.

- الوعود تسهل التعامل مع العمليات **غير المتزامنة** وتجعل الكود أكثر قابلية للقراءة.
- كل وعد يمكن أن يكون في إحدى حالتين نهائيتين: إما **تم الوفاء به** (`resolved`) أو **تم رفضه** (`rejected`).
- نستخدم التابع `.then()` لتنفيذ كود عند نجاح الوعد، و `.catch()` لتنفيذ كود عند فشله.

البرمجة غير المتزامنة — التعامل مع الوعود (then, catch, finally)

التعامل مع الوعود في JavaScript (then, catch, finally)

تُعدّ الوعود (Promises) أساسية في البرمجة غير المتزامنة بـ JavaScript للتعامل مع العمليات التي قد تستغرق وقتًا، مثل جلب البيانات من خادم. تتيح لنا طرق "then", "catch", و "finally" إدارة نتائج هذه العمليات بنجاح أو فشل أو في كلتا الحالتين.

مفهوم الوعود (Promises)

الوعد هو كائن يمثل القيمة النهائية لعملية غير متزامنة، إما نجاحها (Resolved) مع قيمة، أو فشلها (Rejected) مع سبب خطأ. تسمح لنا هذه الطرق بتحديد ما يجب فعله بعد اكتمال الوعد.

مثال عملي

```
function performAsyncOperation(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve("العملية نجحت");
      } else {
        reject("حدث خطأ في العملية");
      }
    }, 1000);
  });
}

// then, catch, finally مثال على استخدام
performAsyncOperation(true) // catch لترى false جرب بـ
  .then(data => {
    console.log("نجاح:", data);
  })
  .catch(error => {
    console.error("خطأ:", error);
  })
  .finally(() => {
```

```
console.log("انتهت العملية سواء نجحت أم فشلت");  
});
```

شرح الكود

في هذا المثال، دالة `performAsyncOperation` تُرجع وعدًا قد ينجح أو يفشل بعد ثانية. يستخدم `then` () للتعامل مع النتيجة عند نجاح الوعد، حيث تستقبل القيمة الممررة من `resolve`. يستخدم `catch` () للتعامل مع الأخطاء عند فشل الوعد، حيث تستقبل سبب الخطأ الممرر من `reject`. وأخيرًا، يتم تنفيذ `finally` () دائمًا بغض النظر عن نتيجة الوعد، لإجراء أي تنظيف ضروري.

- **`then()`:** يُنفذ عند نجاح الوعد (`resolved`) لاستقبال البيانات النهائية.
- **`catch()`:** يُنفذ عند فشل الوعد (`rejected`) لالتقاط الأخطاء ومعالجتها.
- **`finally()`:** يُنفذ دائمًا بعد اكتمال الوعد (سواء نجح أو فشل) لأعمال التنظيف.

البرمجة غير المتزامنة — دوال await و async

دوال await و async

تُعد دوال await و async طريقة حديثة وأنيقة للتعامل مع البرمجة غير المتزامنة في JavaScript، مما يجعل الكود الذي يستخدم الـ Promises أسهل في القراءة والكتابة، وكأنه كود متزامن.

المفهوم الأساسي

الكلمة المفتاحية async تُستخدم لتعريف دالة بأنها غير متزامنة، مما يعني أنها سترجع Promise ضمناً. أما الكلمة المفتاحية await، فنُستخدم داخل الدوال async لإيقاف تنفيذ الدالة مؤقتاً حتى يتم حل Promise معين (إما بنجاح أو فشل)، ثم تستكمل التنفيذ.

مثال برمجي

```
function simulateFetchData() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("!تم جلب البيانات بنجاح");
    }, 2000);
  });
}

async function processData() {
  console.log("...بدء جلب البيانات");
  const data = await simulateFetchData();
  console.log(data);
  console.log("...انتهى المعالجة");
}

processData();
```

شرح الكود

الدالة simulateFetchData تُرجع Promise يتم حلها بعد ثانيتين. الدالة processData هي دالة async. عندما يتم استدعاء await simulateFetchData()، تتوقف الدالة processData عن التنفيذ مؤقتاً وتنتظر حتى يتم حل الـ

Promise من simulateFetchData. بعد حل ال Promise، تستأنف الدالة processData عملها وتطبع الرسائل بالتسلسل المتوقع، مما يجعل تدفق الكود سهلاً للفهم.

- تُحوّل الدالة إلى دالة غير متزامنة تُرجع Promise.
- تُوقف await تنفيذ الدالة async حتى يتم حل ال Promise الذي تنتظره.
- تُبسّط async/await كتابة وقراءة الأكواد التي تعتمد على ال Promises بشكل كبير.

التعامل مع الأخطاء والوحدات — التعامل مع الأخطاء

(try, catch, finally)

التعامل مع الأخطاء (try, catch, finally)

في تطوير التطبيقات، من الضروري توقع الأخطاء التي قد تحدث والتعامل معها بفعالية لضمان استقرار التطبيق وتجربة مستخدم جيدة. توفر جافاسكريبت بنية **try...catch...finally** لإدارة هذه الأخطاء.

مفهوم try, catch, finally

تُستخدم هذه البنية لتنفيذ كود قد يسبب خطأ (try)، والتقاط هذا الخطأ ومعالجته إذا حدث (catch)، ثم تنفيذ كود معين دائماً سواء حدث خطأ أم لا (finally).

مثال عملي

```
function performOperation(value) {  
    try {  
        if (value === 0) {  
            throw new Error("لا يمكن قسمة الرقم على صفر");  
        }  
        let result = 10 / value;  
        console.log("النتيجة:", result);  
    } catch (error) {  
        console.error("حدث خطأ", error.message);  
    } finally {  
        console.log("تم الانتهاء من العملية");  
    }  
}  
  
performOperation(2); // سيعرض النتيجة ثم رسالة الانتهاء  
performOperation(0); // سيلتقط الخطأ ويعرض رسالة الخطأ ثم رسالة الانتهاء
```

شرح الكود

يحتوي الجزء **try** على الكود الذي قد يسبب خطأ، مثل التحقق من القيمة المدخلة. إذا كانت القيمة صفر، يتم إلقاء خطأ يدويًا (throw new Error). الجزء **catch** يلتقط هذا الخطأ ويعرض رسالة الخطأ للمستخدم بطريقة لطيفة. أما الجزء **finally** فينفذ دائمًا بعد انتهاء **try** أو **catch**، لضمان تنفيذ بعض الإجراءات النهائية كإغلاق موارد أو تنظيف.

- يُستخدم **try** لتضمين الكود الذي يُحتمل أن يسبب خطأ.
- يلتقط **catch** الأخطاء التي تحدث داخل **try** ويعالجها بشكل منظم.
- يُنفذ **finally** دائمًا، سواء حدث خطأ أو لم يحدث، وهو مثالي لإجراءات التنظيف.

التعامل مع الأخطاء والوحدات — رمي الأخطاء (throw)

رمي الأخطاء (throw) في JavaScript

في برمجة جافاسكريبت، يعتبر التعامل مع الأخطاء جزءًا حيويًا لبناء تطبيقات قوية وموثوقة. تُستخدم الكلمة المفتاحية `throw` لإنشاء ورمي أخطاء مخصصة عند حدوث شيء غير متوقع أو غير صحيح أثناء تنفيذ البرنامج.

مفهوم الكلمة المفتاحية `throw`

تُستخدم `throw` لإيقاف التدفق الطبيعي للبرنامج والإشارة إلى حدوث خطأ. يمكن رمي أي قيمة كخطأ، سواء كان نصًا، رقمًا، أو كائن `Error`، والأخير هو المفضل دائمًا لتوفير معلومات أفضل عن طبيعة الخطأ.

مثال عملي

```
function validateInput(value) {  
    if (typeof value !== 'number' || value < 0) {  
        throw new Error('القيمة المدخلة غير صحيحة ، يجب أن تكون رقمًا موجبًا');  
    }  
    return 'القيمة صحيحة وتم التحقق منها';  
}  
  
try {  
    console.log(validateInput(10));  
    console.log(validateInput(-5)); // هذا السطر سيطلق خطأ  
} catch (error) {  
    console.error('لقد حدث خطأ', error.message);  
}
```

شرح الكود

في هذا المثال، لدينا دالة `validateInput` تتحقق مما إذا كانت القيمة المدخلة رقمًا موجبًا. إذا لم تكن كذلك، فإنها تستخدم `throw new Error()` لإنشاء ورمي خطأ جديد مع رسالة توضيحية. يتم بعد ذلك التقاط هذا الخطأ بواسطة الكتلة `catch` وعرض رسالته في وحدة التحكم، مما يمنع البرنامج من التوقف بشكل مفاجئ ويسمح بمعالجة الموقف.

نقاط رئيسية

- تُستخدم **throw** لإنشاء ورمي الأخطاء المخصصة في JavaScript.
- عند رمي خطأ، يتوقف التنفيذ الطبيعي ويجب معالجته باستخدام كتلة **try...catch**.
- رمي الأخطاء يساعد في الإبلاغ عن المشكلات وجعل التعليمات البرمجية أكثر قوة وقابلية للصيانة.

التعامل مع الأخطاء والوحدات — الوحدات (Modules)

الوحدات (Modules) في JavaScript

تساعدنا الوحدات في تنظيم شيفرة JavaScript الخاصة بنا وتقسيمها إلى ملفات أصغر ومستقلة. هذا يجعل الشيفرة أسهل في الفهم، الصيانة، وإعادة الاستخدام، ويعزز هيكلية التطبيقات الكبيرة.

مفهوم الوحدات

الوحدات هي طريقة لتقسيم برنامجك إلى أجزاء منفصلة. كل جزء (ملف) يمكن أن يُصدّر (export) وظائف أو متغيرات ليتم استخدامها في أجزاء أخرى، أو أن يستورد (import) وظائف ومتغيرات من وحدات أخرى.

مثال عملي

```
// math.js (الوحدة المصدرة)
export function add(a, b) {
    return a + b;
}

export const PI = 3.14159;

// app.js (الوحدة المستوردة)
import { add, PI } from './math.js';

console.log("الجمع:", add(5, 3));
console.log("PI: قيمة", PI);
```

شرح الكود

في ملف math.js، قمنا بتصدير الدالة add والمتغير PI باستخدام الكلمة المفتاحية export. في ملف app.js، استوردنا هذه العناصر المصدرة من 'math.js/' باستخدام الكلمة المفتاحية import، ثم استخدمناها بكل سهولة في برنامجنا لطباعة النتائج.

- **تنظيم الشيفرة:** تجعل الوحدات شيفرتك أكثر ترتيبًا وسهولة في الإدارة والصيانة.

- **إعادة الاستخدام:** يمكنك إعادة استخدام الوظائف والمتغيرات المصدرة في أجزاء مختلفة من مشروعك دون تكرار.
- **منع التعارض:** تساعد في تجنب تعارض الأسماء بين المتغيرات والدوال المختلفة لأن لكل وحدة نطاقها الخاص.

التعامل مع الأخطاء والوحدات — الاستيراد والتصدير (import/export)

الاستيراد والتصدير (import/export)

تعتبر الوحدات (Modules) في جافاسكريبت طريقة رائعة لتنظيم الكود الخاص بك وتقسيمه إلى ملفات منفصلة وأكثر قابلية للإدارة. تتيح لك آليتا الاستيراد (**import**) والتصدير (**export**) مشاركة الوظائف والمتغيرات والكلاسات بين هذه الملفات، مما يعزز إعادة الاستخدام ويجعل مشروعك أكثر تنظيماً.

مفهوم الاستيراد والتصدير

التصدير (export): تُستخدم لجعل أجزاء من الكود (مثل الدوال أو المتغيرات) متاحة للاستخدام في ملفات جافاسكريبت أخرى.

الاستيراد (import): تُستخدم لجلب الأجزاء التي تم تصديرها من ملف آخر إلى الملف الحالي لاستخدامها.

مثال عملي

```
math.js: في الملف //
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;

app.js: في الملف //
import { add, subtract } from './math.js';

console.log(add(10, 5));
console.log(subtract(20, 7));
```

شرح الكود

في هذا المثال، لدينا ملفان. الملف الأول (تخيل أنه **math.js**) يقوم **بتصدير** دالتين: **add** و **subtract**. استخدم الكلمة المفتاحية **export** يجعلهما متاحين للاستخدام في ملفات أخرى. أما الملف الثاني (تخيل أنه **app.js**) فيقوم **باستيراد** هاتين الدالتين من **math.js** باستخدام **import { add, subtract } from './math.js'**. بعد الاستيراد، يمكن استخدام الدوال **add** و **subtract** كما لو كانت معرفة مباشرة في **app.js** لجمع وطرح الأرقام.

نقاط رئيسية

- **تنظيم الكود:** تقسيم المشروع إلى ملفات صغيرة ومنظمة يسهل قراءته وصيانته.
- **إعادة الاستخدام:** يمكنك كتابة جزء من الكود مرة واحدة وتصديره، ثم استيراده واستخدامه في أي مكان آخر بالمشروع.
- **الاعتمادية الواضحة:** يوضح نظام الاستيراد والتصدير بشكل مباشر أي الأجزاء تعتمد على أجزاء أخرى، مما يحسن من فهم بنية المشروع.